

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE
PETRÓLEO
CENTRO DE CIÊNCIA E TECNOLOGIA

PEC - Pore Elastic Calculator
Ferramenta Computacional para Simulação de Propriedades
Poroelásticas em Rochas Reservatório

DISCIPLINA LEP 01449: Projeto de Software Aplicado à Engenharia
Setor de Modelagem Matemática Computacional

Versão 3.0

Matheus Sousa Bastos
Nicolau Azevedo Prates

Prof. André Duarte Bueno

MACAÉ - RJ
Dezembro - 2025

Sumário

1	Introdução	5
1.1	Escopo do problema	5
1.2	Objetivos	5
1.3	Metodologia utilizada	6
2	Concepção	8
2.1	Metodologia	8
2.2	Nome do Sistema/Produto	9
2.3	Especificação	9
2.4	Requisitos	9
2.4.1	Requisitos funcionais	9
2.4.2	Requisitos não funcionais	10
2.5	Casos de Uso	10
2.5.1	Diagrama de caso de uso geral	10
2.5.2	Diagrama de caso de uso específico	10
3	Elaboração	12
3.1	Análise de domínio	12
3.2	Formulação teórica	12
3.2.1	Modelo de Voigt-Reuss-Hill (VRH)	12
3.2.2	Limites de Hashin-Shtrikman (HS)	13
3.3	Identificação de pacotes – assuntos	13
3.4	Diagrama de pacotes – assuntos	14
4	AOO – Análise Orientada a Objeto	15
4.1	Diagramas de classes	15
4.1.1	Dicionário de classes	15
4.2	Diagrama de seqüência – eventos e mensagens	21
4.2.1	Diagrama de seqüência geral	21
4.2.2	Diagrama de seqüência específico	23
4.3	Diagrama de comunicação – colaboração	23
4.4	Diagrama de estado	24
4.5	Diagrama de atividades	25

5	Projeto	27
5.1	Projeto do Sistema	27
5.2	Projeto Orientado a Objeto – POO	29
5.3	Diagrama de Componentes	32
5.4	Diagrama de Implantação	34
5.4.1	Lista de características «features»	35
6	Ciclos de Planejamento/Detalhamento	37
6.1	Lista de características «features»	37
7	Ciclos Construção - Implementação	39
7.1	Configuração e Scripts Auxiliares	39
7.2	Interfaces do Sistema (.h)	40
7.2.1	Entidades Básicas	40
7.2.2	Gestão de Dados e Entrada	41
7.2.3	Motor de Cálculo (Física)	45
7.2.4	Visualização e Controle	47
7.3	Implementação (.cpp)	49
7.3.1	Entidades do Domínio	49
7.3.2	Gestão de Dados e Entrada (I/O)	51
7.3.3	Motor de Cálculo (Física de Rochas)	59
7.3.4	Visualização e Orquestração	60
8	Testes	69
8.1	Metodologia e Ambiente de Teste	69
8.2	Teste de Interface e Entrada Manual	70
8.3	Validação Numérica e Processamento em Lote	70
8.4	Análise de Sensibilidade e Visualização Gráfica	71
8.5	Conclusão dos Testes	72
9	Documentação para o Desenvolvedor	73
9.1	Dependências e Ambiente de Desenvolvimento	73
9.2	Sugestões para Trabalhos Futuros	74
	Referências Bibliográficas	76

Lista de Figuras

2.1	Metodologia utilizada no desenvolvimento do sistema	8
2.2	Diagrama de caso de uso – Caso de uso geral	10
2.3	Diagrama de caso de uso específico	11
3.1	Diagrama de Pacotes	14
4.1	Diagrama de classes	16
4.2	Diagrama de sequência geral	22
4.3	Diagrama de sequência específico	23
4.4	Diagrama de comunicação	24
4.5	Diagrama de máquina de estado	24
4.6	Diagrama de atividades	26
5.1	Diagrama de componentes	33
5.2	Diagrama de implantação	34
8.1	Interface principal do sistema PEC demonstrando o menu interativo e a prontidão para receber comandos do usuário.	70
8.2	Resumo consolidado das amostras processadas, exibindo os valores calculados para os módulos de compressibilidade e cisalhamento.	71
8.3	Gráfico de análise de sensibilidade gerado pelo PEC, ilustrando a variação dos módulos elásticos (limites de Voigt-Reuss-Hill) em função da composição mineralógica.	72

Lista de Tabelas

2.1	Caso de uso 1	10
-----	-------------------------	----

Capítulo 1

Introdução

A caracterização petrofísica e geomecânica de rochas reservatório é uma etapa crítica na indústria de exploração e produção de petróleo. A capacidade de prever o comportamento elástico da rocha — especificamente sua resistência à compressão e ao cisalhamento — a partir de dados mineralógicos básicos é fundamental para a interpretação sísmica e para o planejamento de estabilidade de poços. Neste contexto, apresenta-se o projeto do software **PoreElasticCalculator (PEC)**, uma ferramenta computacional desenvolvida para automatizar a modelagem de física de rochas através da Teoria do Meio Efetivo.

1.1 Escopo do problema

O escopo deste projeto define-se pela necessidade de estimar propriedades elásticas macroscópicas em meios porosos heterogêneos onde medições diretas de laboratório são escassas, custosas ou inexistentes. O problema central consiste em determinar os módulos elásticos efetivos — Módulo de Compressibilidade (K) e Módulo de Cisalhamento (G) — de uma rocha composta por uma mistura arbitrária de minerais e uma fase de porosidade.

O projeto limita-se à modelagem estática de meios isotrópicos utilizando as médias clássicas de Voigt, Reuss e Hill. O software aborda rochas clásticas e carbonáticas, permitindo análises de sensibilidade mineralógica (ex: impacto do aumento de argila na rigidez da rocha). Não escopo deste trabalho a simulação dinâmica de propagação de ondas, a modelagem de anisotropia ou efeitos de frequência dependente (dispersão), focando estritamente na aplicação dos limites elásticos teóricos para a caracterização rápida de perfis de poço e amostras de laboratório.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver uma ferramenta de engenharia robusta, modular e multiplataforma, implementada na linguagem C++, capaz de realizar

cálculos de física de rochas com precisão e eficiência.

Para alcançar este propósito, foram definidos os seguintes objetivos específicos:

- **Modelagem Matemática:** Implementar computacionalmente os algoritmos das médias de Voigt (limite superior de rigidez), Reuss (limite inferior) e Hill (média aritmética) para compósitos de N fases minerais mais porosidade.
- **Arquitetura de Software:** Desenvolver um sistema orientado a objetos (POO) extensível, utilizando padrões de projeto (*Strategy*, *Facade*) para desacoplar a interface de usuário, o motor de cálculo e a camada de dados.
- **Automação e Produtividade:** Criar funcionalidades para processamento em lote (leitura de arquivos de múltiplas amostras) e persistência de dados (banco de minerais editável em CSV).
- **Visualização de Dados:** Integrar o software com a ferramenta Gnuplot para a geração automática de gráficos de alta qualidade, permitindo a visualização de perfis de propriedades por profundidade e correlações cruzadas (ex: K vs. Porosidade).
- **Portabilidade:** Garantir a compilação e execução do software em diferentes sistemas operacionais (Windows, Linux, macOS) através do sistema de build CMake.

1.3 Metodologia utilizada

A metodologia adotada para o desenvolvimento do PEC combinou fundamentos da Engenharia de Software com a Física de Rochas Teórica.

Fundamentação Teórica

O núcleo físico do software baseia-se na Teoria do Meio Efetivo (EMT). A rocha é tratada como um material compósito onde a porosidade é matematicamente modelada como uma inclusão mineral com módulos elásticos nulos ($K \approx 0, G \approx 0$). O algoritmo normaliza as frações volumétricas da matriz sólida e aplica as equações de médias harmônicas e aritméticas para obter os módulos efetivos.

Implementação Computacional

O desenvolvimento seguiu um modelo iterativo e incremental, utilizando as seguintes tecnologias e práticas:

- **Linguagem de Programação:** C++ (padrão C++17) foi escolhido pela alta performance e suporte robusto à orientação a objetos. Utilizou-se extensivamente

a Biblioteca Padrão (STL) para gestão de memória e estruturas de dados (vetores, mapas).

- **Ambiente de Build:** O CMake foi utilizado para gerenciar o processo de compilação, assegurando que o código fosse agnóstico ao sistema operacional e ao compilador (GCC, Clang, MSVC).
- **Visualização:** A geração de gráficos foi implementada através de um pipeline de scripting dinâmico. A classe `CPlotManager` gera scripts `.gp` em tempo de execução e invoca o Gnuplot como um processo subprocesso do sistema, garantindo flexibilidade e qualidade gráfica.
- **Gestão de Configuração:** O controle de versão foi realizado via Git/GitHub, permitindo o rastreamento de alterações e a gestão de diferentes versões do código (núcleo de cálculo, interface, testes).

Concepção

Apresenta-se neste capítulo do projeto de engenharia a concepção, a especificação do sistema a ser modelado e desenvolvido.

2.1 Metodologia

A Figura 2.1 apresenta a metodologia a ser utilizada no desenvolvimento do sistema.

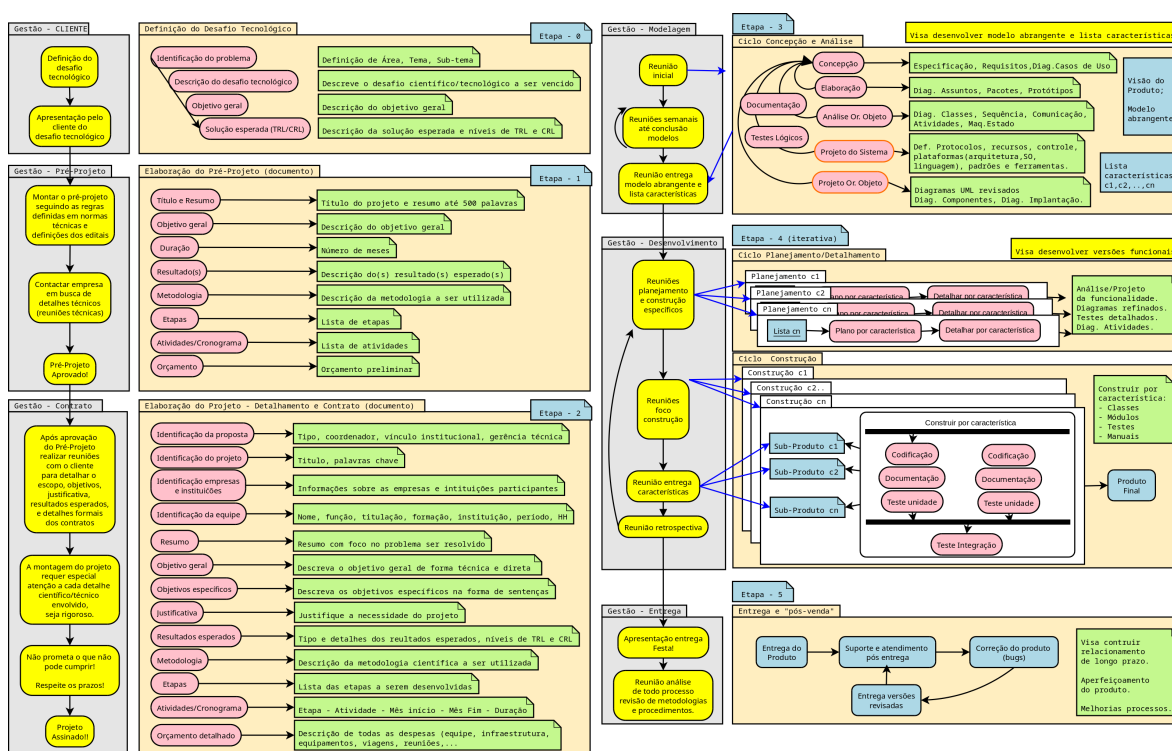


Figura 2.1: Metodologia utilizada no desenvolvimento do sistema

2.2 Nome do Sistema/Produto

Nome	PoroElasticCalculator
Componentes principais	MineralDatabase, ReservoirData, PECAApplication
Missão	Calcular propriedades elásticas de amostras de rocha

2.3 Especificação

O software PEC (PoroElasticCalculator) tem como objetivo estimar as propriedades elásticas efetivas de rochas sedimentares a partir de sua composição mineralógica, por meio da aplicação de modelos físicos consagrados na literatura de física das rochas.

O sistema visa auxiliar engenheiros, geofísicos e pesquisadores na avaliação de módulo de compressibilidade (K) e módulo de cisalhamento (G) de amostras naturais ou sintéticas.

2.4 Requisitos

Apresenta-se nesta seção os requisitos funcionais e não funcionais.

2.4.1 Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

RF-01	O sistema deve conter uma base de dados dos módulos de cisalhamento e compressibilidade de vários minerais.
RF-02	O usuário deverá ter liberdade para colocar manualmente os valores de sua amostra ou carregar um arquivo de saída que já contenha estes dados.
RF-03	Deve permitir o carregamento de arquivos criados pelo software.
RF-04	Deve permitir a escolha das equações de Hashin-Strikeman ou Voig-Reus-Hill.
RF-05	O O usuário deve ter tal liberdade para escolher as porcentagens mineralógicas da sua amostra.
RF-06	O usuário poderá plotar seus resultados em um gráfico. O gráfico poderá ser salvo como imagem ou ter seus dados exportados como texto.

2.4.2 Requisitos não funcionais

RNF-01	Os cálculos devem ser feitos utilizando-se as equações de Hashin-Strikeman ou Voig-Reus-Hill.
RNF-02	O programa deverá ser multi-plataforma, podendo ser executado em <i>Windows</i> , <i>GNU/Linux</i> ou <i>MacOS</i> .

2.5 Casos de Uso

A Tabela 2.1 mostra a descrição de um caso de uso.

Tabela 2.1: Caso de uso 1

Nome do caso de uso:	Geral
Resumo/descrição:	Etapas a serem cumpridas pelo usuário
Etapas:	Criar a amostra, carregar os arquivos, gerar os crossplot, analisar os resultados.
Cenários alternativos:	O usuário escolhe os valores manualmente, podendo adicionar novos minerais

2.5.1 Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 2.2 mostra o usuário acessando os sistemas de ajuda do software, calculando as propriedades elásticas e analisando os resultados.

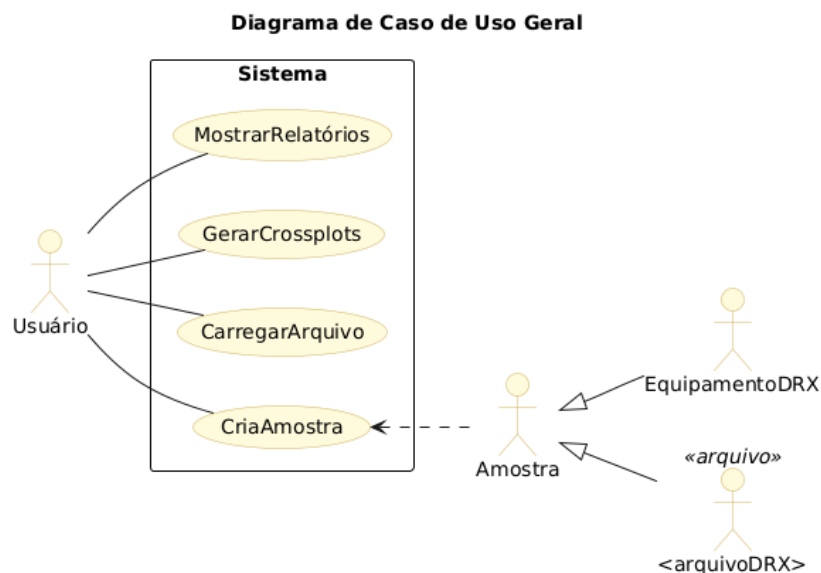


Figura 2.2: Diagrama de caso de uso – Caso de uso geral

2.5.2 Diagrama de caso de uso específico

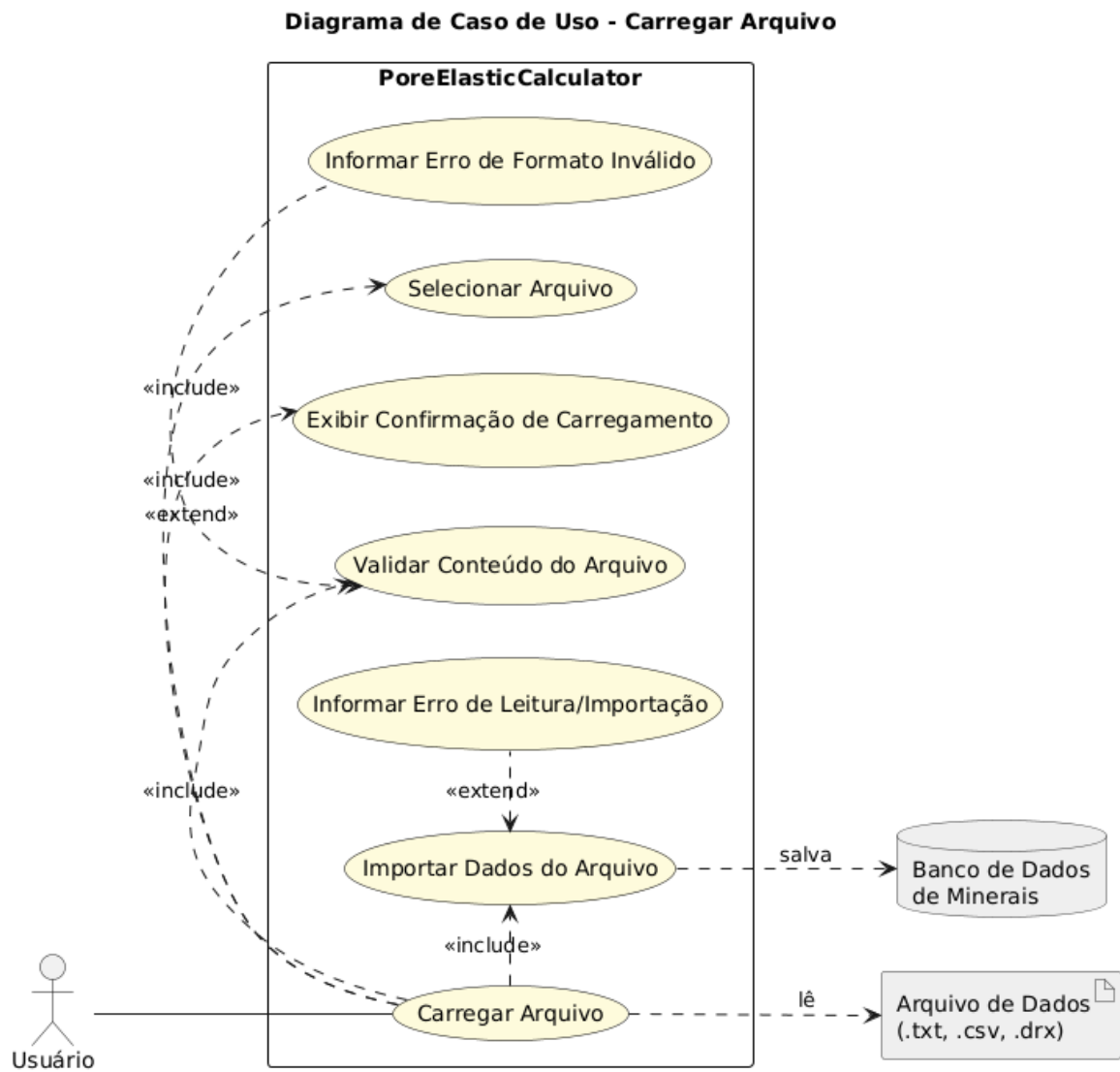


Figura 2.3: Diagrama de caso de uso específico

Capítulo 3

Elaboração

3.1 Análise de domínio

Após estudo dos requisitos/especificações do sistema, algumas entrevistas, estudos na biblioteca e disciplinas do curso foi possível identificar nosso domínio de trabalho:

- Área de aplicação: Engenharia de Petróleo, Petrofísica, Geomecânica
- Problema central: Determinar propriedades elásticas (módulo de cisalhamento e compressibilidade) de rochas com composições mineralógicas variadas, algo comum em formações sedimentares.

3.2 Formulação teórica

A determinação das propriedades elásticas efetivas de rochas com múltiplos constituintes minerais exige o uso de modelos de homogeneização que aproximem o comportamento macroscópico do meio composto. Neste trabalho, são utilizados os modelos de Voigt-Reuss-Hill (VRH) e os limites de Hashin-Shtrikman (HS) para a estimativa do módulo de compressibilidade K e do módulo de cisalhamento G da rocha.

3.2.1 Modelo de Voigt-Reuss-Hill (VRH)

O modelo de Voigt assume deformação uniforme entre os constituintes da mistura, resultando em uma média aritmética ponderada das propriedades dos minerais:

$$K_V = \sum_{i=1}^n f_i K_i \quad (3.1)$$

$$G_V = \sum_{i=1}^n f_i G_i \quad (3.2)$$

Já o modelo de Reuss considera tensão uniforme entre os constituintes, levando a uma média harmônica ponderada:

$$\frac{1}{K_R} = \sum_{i=1}^n \frac{f_i}{K_i} \quad (3.3)$$

$$\frac{1}{G_R} = \sum_{i=1}^n \frac{f_i}{G_i} \quad (3.4)$$

A média de Voigt-Reuss-Hill (VRH) é obtida como a média aritmética simples entre os valores de Voigt e Reuss:

$$K_{VRH} = \frac{K_V + K_R}{2} \quad (3.5)$$

$$G_{VRH} = \frac{G_V + G_R}{2} \quad (3.6)$$

3.2.2 Limites de Hashin-Shtrikman (HS)

Os limites de Hashin-Shtrikman oferecem uma estimativa teórica rigorosa dos valores mínimo e máximo possíveis para as propriedades elásticas de materiais isotrópicos compostos por duas ou mais fases. Supondo K_1 e G_1 como os módulos do mineral com maior rigidez e K_2 e G_2 como os módulos do mineral mais fraco, com frações volumétricas f_1 e f_2 , os limites são dados por:

$$K_{HS}^+ = K_2 + \frac{f_1}{1K_1 - K_2 + f_2K_2 + 43G_2} \quad (3.7)$$

$$G_{HS}^+ = G_2 + \frac{f_1}{1G_1 - G_2 + f_2G_2 + \zeta} \quad (3.8)$$

$$K_{HS}^- = K_1 + \frac{f_2}{1K_2 - K_1 + f_1K_1 + 43G_1} \quad (3.9)$$

$$G_{HS}^- = G_1 + \frac{f_2}{1G_2 - G_1 + f_1G_1 + \zeta'} \quad (3.10)$$

Onde:

$$\zeta = \frac{G_2(9K_2 + 8G_2)}{6(K_2 + 2G_2)} \quad ; \quad \zeta' = \frac{G_1(9K_1 + 8G_1)}{6(K_1 + 2G_1)} \quad (3.11)$$

Esses limites fornecem uma faixa possível para as propriedades efetivas, sendo úteis para validar os resultados obtidos com modelos mais simplificados como VRH.

3.3 Identificação de pacotes – assuntos

Este projeto fundamenta-se em um conjunto de pacotes teóricos e computacionais voltados para o estudo das propriedades elásticas de rochas com múltiplas composições mineralógicas. A seguir são descritos os principais pacotes envolvidos, com sua respectiva função no desenvolvimento do modelo.

- **Rock Physics:** Conjunto de teorias que conectam propriedades elásticas da rocha (como módulo de cisalhamento e compressibilidade) com suas características petrofísicas, como composição mineralógica, porosidade e saturação. Este pacote fornece a base conceitual para a formulação dos modelos de mistura empregados neste trabalho.
- **Mineral Physics:** Ramo que estuda as propriedades físicas dos minerais puros, como o módulo de compressibilidade (K) e o módulo de cisalhamento (G). Esses valores são fundamentais como dados de entrada para os modelos de homogeneização. Os dados típicos são obtidos da literatura especializada.
- **Método de homogeneização** que fornece uma média das propriedades elásticas baseada em hipóteses de deformação (Voigt) e tensão (Reuss) uniformes. A média de Voigt-Reuss-Hill representa uma estimativa intermediária entre os extremos teóricos.
- **Gnuplot**, ferramenta de visualização utilizada para representar graficamente os resultados dos modelos, como comparação entre VRH e HS, análises de sensibilidade e tendências com variação mineralógica.
- **Teoria dos meios efetivos:** Pacote teórico que abrange todos os modelos de homogeneização para materiais compostos, como VRH, HS, DEM (Differential Effective Medium) e outros. Justifica conceitualmente o uso dos modelos implementados neste trabalho.

3.4 Diagrama de pacotes – assuntos

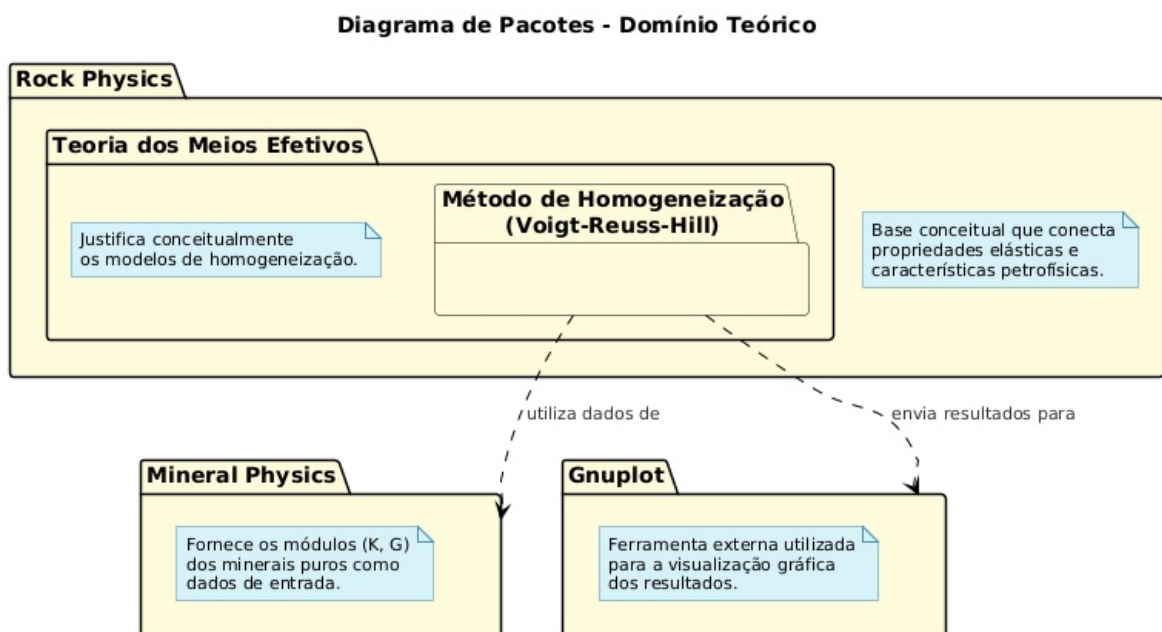


Figura 3.1: Diagrama de Pacotes

Capítulo 4

AOO – Análise Orientada a Objeto

4.1 Diagramas de classes

O diagrama de classes é apresentado na Figura 4.1.

4.1.1 Dicionário de classes

- **CSimulator**

Responsabilidade: Classe principal que orquestra a execução do programa.

É responsável por inicializar todos os componentes, gerenciar o menu de interação com o usuário e controlar o fluxo de dados entre os módulos de entrada, cálculo e visualização.

Atributos: – `-m_input`: `CInput` – Instância do gerenciador de entrada de dados.

– `-m_database`: `CMineralDatabase` – Instância do banco de dados de minerais.

– `-m_calculator`: `CElasticCalculator` – Instância do módulo de cálculo de propriedades elásticas.

– `-m_reservoirData`: `CReservoirData` – Instância do repositório de dados das amostras processadas.

– `-m_plotManager`: `CPlotManager` – Instância do gerenciador de visualização gráfica.

Métodos: – `+CSimulator()` / `+~CSimulator()`: Construtor e destrutor da classe.

– `+Run()`: Inicia e gerencia o laço de execução principal da aplicação.

– `-Initialize()`: Prepara e aloca os recursos para os módulos do sistema.

– `-Menu()`: Exibe as opções disponíveis e captura a escolha do usuário.

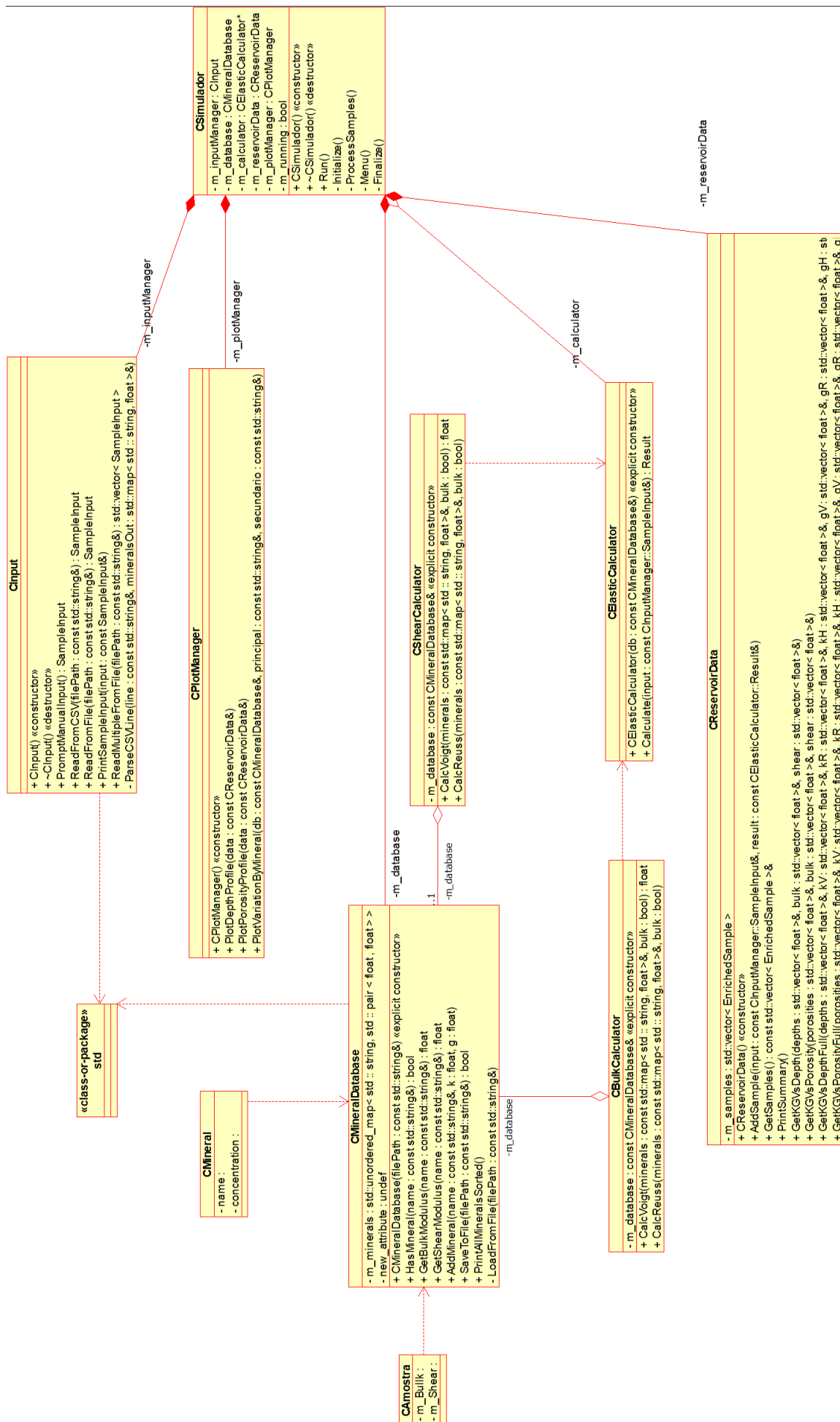


Figura 4.1: Diagrama de classes

- `-ProcessSamples()`: Dispara o fluxo de processamento para uma nova amostra.
- `-Finalize()`: Libera os recursos alocados antes de encerrar a aplicação.

- **CInput**

Responsabilidade: Gerencia a aquisição de dados das amostras, seja por meio de entrada manual do usuário no console ou pela leitura e interpretação de arquivos de dados formatados.

Métodos: – `+CInput()` / `+~CInput()`: Construtor e destrutor da classe.

- `+PromptManualInput()`: Solicita interativamente os dados da amostra ao usuário.
- `+ReadFromCSV()`: Lê dados de um arquivo no formato CSV (não utilizado na versão final).
- `+ReadFromFile()`: Lê os dados de uma única amostra de um arquivo de texto.
- `+PrintSampleInput()`: Exibe os dados de entrada de uma amostra no console para verificação.
- `+ReadMultipleFromFile()`: Lê uma ou mais amostras de um arquivo de texto.
- `-ParseCSVLine()`: Interpreta uma única linha de um arquivo de dados para extrair a composição mineralógica.

- **CMineralDatabase**

Responsabilidade: Encapsula o acesso à base de dados de minerais. É responsável por carregar, salvar e fornecer as propriedades elásticas (módulo de compressibilidade e de cisalhamento) de cada mineral.

Atributos: – `-m_minerals`: Estrutura de dados que armazena as propriedades dos minerais carregados em memória.

- `-m_databaseFilePath`: Caminho para o arquivo físico da base de dados.

Métodos: – `+CMineralDatabase(filePath)`: Construtor que inicializa o objeto e carrega os dados do arquivo especificado.

- `+HasMineral(name)`: Verifica a existência de um mineral na base de dados.
- `+GetBulkModulus(name)`: Retorna o módulo de compressibilidade (K) de um mineral.
- `+GetShearModulus(name)`: Retorna o módulo de cisalhamento (G) de um mineral.

- `+AddMineral(name, k, g)`: Adiciona um novo mineral à base de dados em memória.
- `+SaveToFile(filePath)`: Salva o estado atual da base de dados em um arquivo.
- `+PrintAllMineralsSorted()`: Exibe todos os minerais da base de dados em ordem alfabética.
- `-LoadFromFile(filePath)`: Lógica interna para ler e interpretar o arquivo da base de dados.

- **CElasticCalculator**

Responsabilidade: Constitui o núcleo de cálculo do sistema. Implementa os modelos de homogeneização de Voigt e Reuss para estimar os módulos elásticos efetivos da matriz rochosa.

Atributos: – `-m_database: CMineralDatabase&` – Referência para a base de dados de minerais, utilizada para consultar as propriedades durante o cálculo.

Métodos: – `+CElasticCalculator(db)`: Construtor que recebe uma referência ao banco de dados de minerais.

- `+Calculate(input)`: Orquestra o cálculo para uma amostra, invocando os métodos de Voigt e Reuss e retornando os resultados.
- `-CalcVoigt(minerals, bulk)`: Implementa a média de Voigt para o módulo de compressibilidade ou de cisalhamento.
- `-CalcReuss(minerals, bulk)`: Implementa a média de Reuss para o módulo de compressibilidade ou de cisalhamento.

- **CReservoirData**

Responsabilidade: Atua como um repositório para as amostras processadas. Armazena os dados de entrada junto com os resultados calculados para uso posterior em relatórios e gráficos.

Atributos: – `-m_samples`: Vetor que armazena o conjunto de todas as amostras processadas durante a sessão.

Métodos: – `+CReservoirData()`: Construtor da classe.

- `+AddSample(input, result)`: Adiciona uma nova amostra e seus resultados calculados ao repositório.
- `+GetSamples()`: Retorna uma referência para o vetor completo de amostras.
- `+PrintSummary()`: Exibe no console um resumo com os principais resultados de todas as amostras.

- `+GetKGVvsDepth()`: Extrai e organiza os dados para plotagem de propriedades vs. profundidade.
- `+GetKGVvsPorosity()`: Extrai e organiza os dados para plotagem de propriedades vs. porosidade.
- `+GetKGVvsDepthFull()`: Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de profundidade.
- `+GetKGVvsPorosityFull()`: Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de porosidade.

- **CPlotManager**

Responsabilidade: Módulo dedicado à visualização de dados. Utiliza os resultados consolidados em `CReservoirData` para gerar e exibir gráficos, facilitando a análise e interpretação dos resultados.

Métodos: – `+CPlotManager()`: Construtor da classe.

- `+PlotDepthProfile(data)`: Gera o gráfico dos módulos elásticos em função da profundidade para as amostras fornecidas.
- `+PlotPorosityProfile(data)`: Gera o gráfico dos módulos elásticos em função da porosidade.
- `+PlotVariationByMineral(db, ...)`: Cria gráficos de sensibilidade à variação percentual entre dois minerais.

- **CAmostra**

Responsabilidade: Encapsula todos os dados pertinentes a uma única amostra de rocha. Funciona como uma estrutura de dados para transportar informações sobre a amostra através do sistema.

Atributos: – `-m_reservatorio`: Armazena o nome do reservatório de onde a amostra foi extraída.

- `-m_nome`: Identificador único da amostra.
- `-m_profundidade`: Profundidade (em metros) na qual a amostra foi coletada.
- `-m_porosidade`: Valor percentual da porosidade da rocha.
- `-m_minerais`: Estrutura de dados (ex.: mapa) que associa o nome de cada mineral à sua proporção na amostra.

Métodos: – `+CAmostra(...)`: Construtor que inicializa todos os atributos do objeto.

- `+NomeReservatorio()`: Retorna o nome do reservatório.
- `+NomeAmostra()`: Retorna o nome da amostra.
- `+Profundidade()`: Retorna o valor da profundidade.

- `+Porosidade()`: Retorna o valor da porosidade.
- `+Minerais()`: Retorna a coleção de minerais e suas respectivas proporções.

- **CMineral**

Responsabilidade: Representa a entidade de um mineral, contendo suas propriedades elásticas fundamentais.

Atributos: – `-m_name`: O nome do mineral (ex.: Quartzo, Albite).

- `-m_bulkModulus`: Módulo de compressibilidade (K) do mineral, em GPa.
- `-m_shearModulus`: Módulo de cisalhamento (G) do mineral, em GPa.

Métodos: – `+CMineral(name, k, g)`: Construtor que inicializa o objeto com os dados do mineral.

- `+Name()`: Retorna o nome do mineral.
- `+BulkModulus()`: Retorna o módulo de compressibilidade (K).
- `+ShearModulus()`: Retorna o módulo de cisalhamento (G).

- **CShearCalculator**

Responsabilidade: Especializada no cálculo do módulo de cisalhamento (G) efetivo de uma amostra de rocha, com base em sua composição mineralógica.

Atributos: – `-m_db`: Referência à `CMineralDatabase`, utilizada para consultar propriedades de cisalhamento dos minerais individuais.

Métodos: – `+CShearCalculator(db)`: Construtor que recebe a base de dados de minerais.

- `+CalculateVoigt(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Voigt (limite superior).
- `+CalculateReuss(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Reuss (limite inferior).
- `+CalculateHill(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Hill (média entre Voigt e Reuss).

- **CBulkCalculator**

Responsabilidade: Especializada no cálculo do módulo de compressibilidade (K) efetivo de uma amostra de rocha, com base em sua composição mineralógica.

Atributos: – `-m_db`: Referência à `CMineralDatabase`, utilizada para consultar propriedades de compressibilidade dos minerais individuais.

- Métodos:**
- `+CBulkCalculator(db)`: Construtor que recebe a base de dados de minerais.
 - `+CalculateVoigt(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Voigt.
 - `+CalculateReuss(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Reuss.
 - `+CalculateHill(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Hill.

4.2 Diagrama de seqüência – eventos e mensagens

4.2.1 Diagrama de sequência geral

Veja o diagrama de seqüência na Figura 4.2.

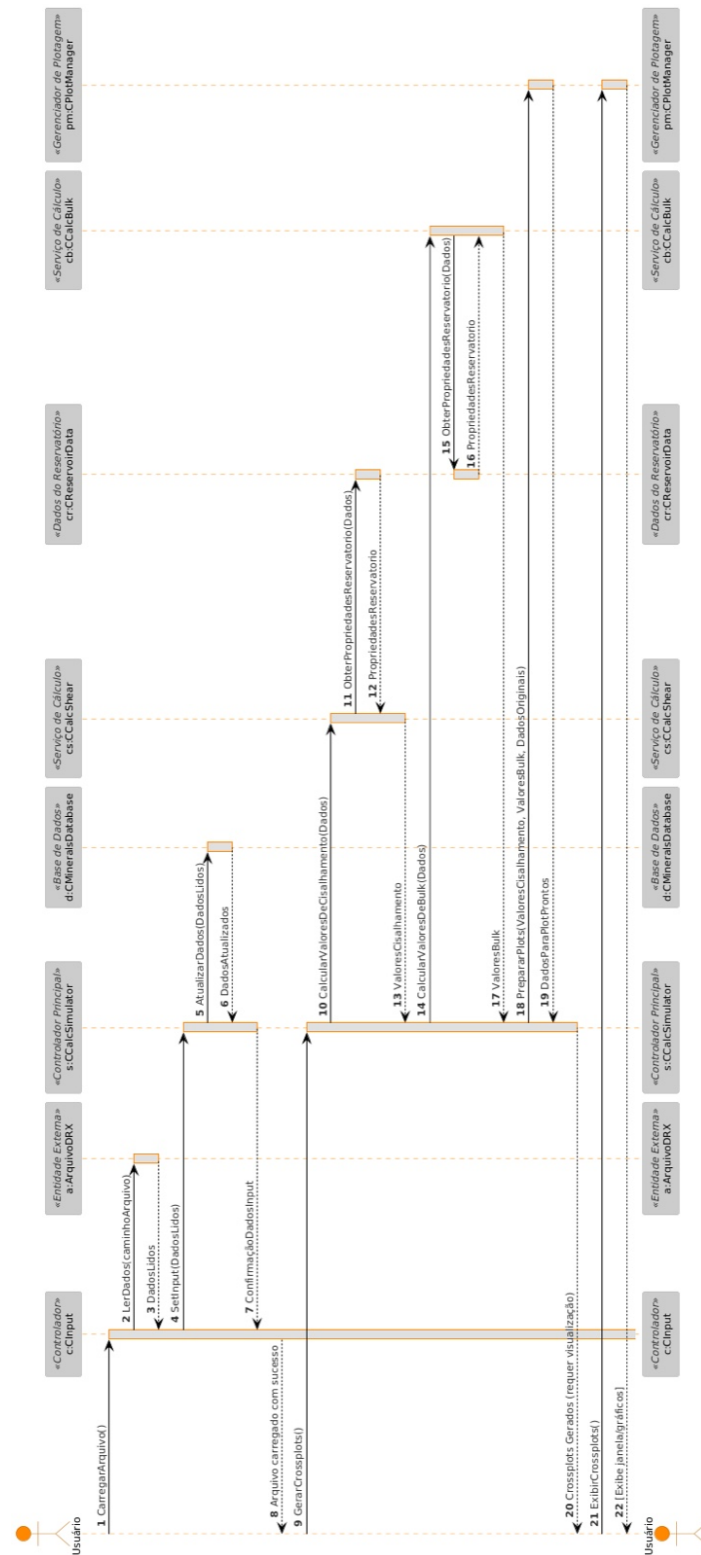


Figura 4.2: Diagrama de sequência geral

4.2.2 Diagrama de sequência específico

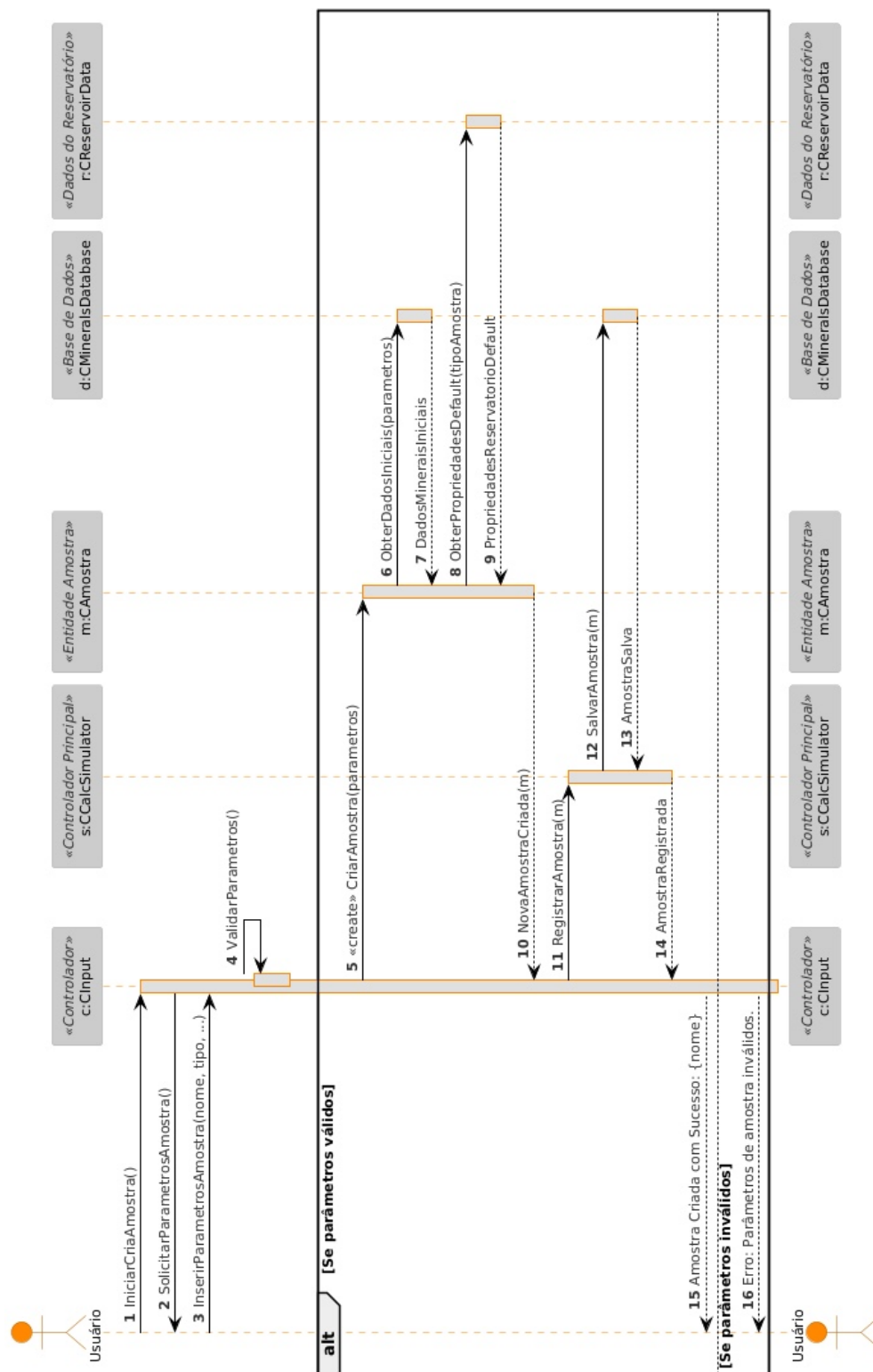


Figura 4.3: Diagrama de sequência específico

4.3 Diagrama de comunicação – colaboração

No diagrama de comunicação o foco é a interação e a troca de mensagens e dados entre os objetos.

Veja na Figura 4.3 o diagrama de comunicação mostrando a sequência de interação, onde a aplicação solicita leitura do arquivo da amostra para dar prosseguimento com o cálculo das propriedades, onde busca os dados do banco de dados. A aplicação armazena os dados e gera os resultados, e por fim solicita a geração dos gráficos de perfil.

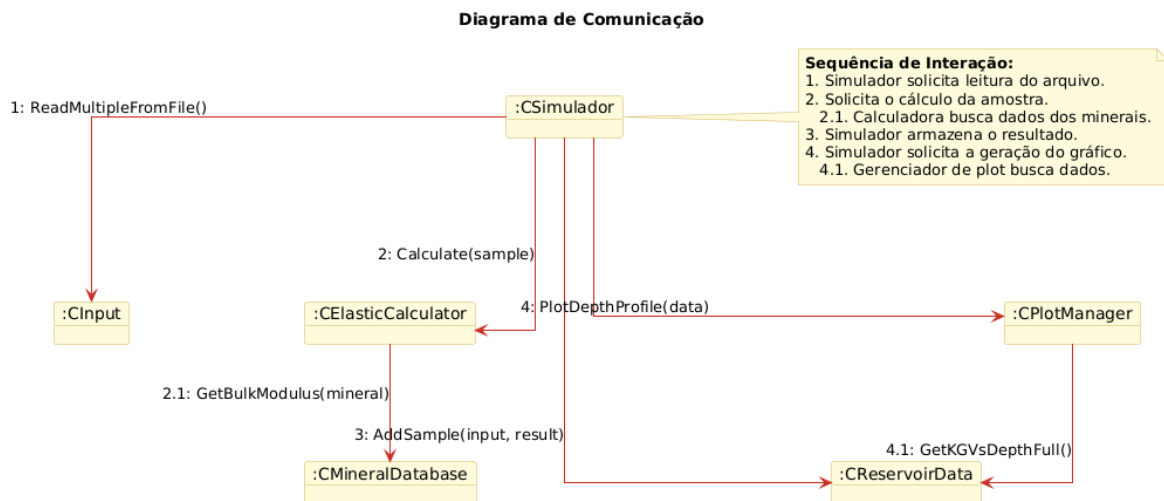


Figura 4.4: Diagrama de comunicação

4.4 Diagrama de estado

Veja na Figura 4.5 o diagrama de máquina de estado para o objeto CSimulador

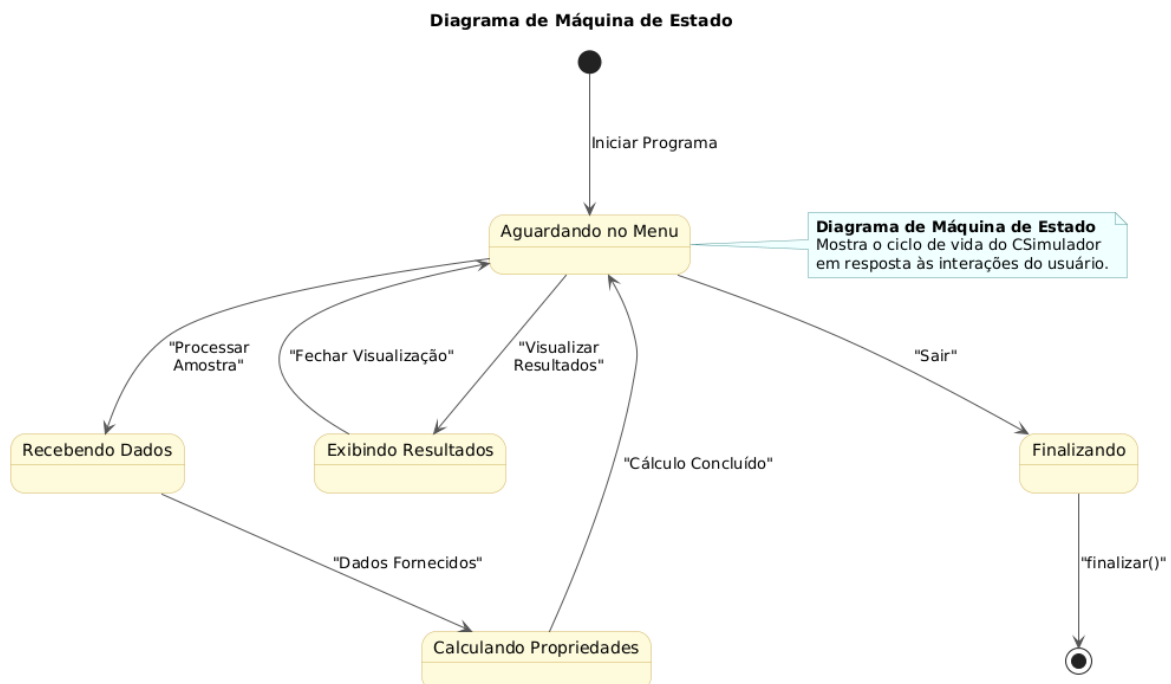


Figura 4.5: Diagrama de máquina de estado

4.5 Diagrama de atividades

Veja na Figura 4.6 o diagrama de atividades.

O processo tem início no Nó Inicial e transita para um laço de repetição, o qual representa o ciclo de vida interativo do menu principal e se mantém ativo enquanto a condição de controle `m_running` for verdadeira. Dentro deste laço, a primeira Atividade consiste em "Exibir Opções do Menu" no console, seguida imediatamente pela Atividade "Ler a escolha do usuário".

Ato contínuo, o fluxo chega a um Nó de Decisão, implementado como uma estrutura `switch`, que direciona o controle com base na opção fornecida. Se o usuário escolher as opções '1' ("Inserir nova amostra manualmente") ou '2' ("Carregar amostra de arquivo"), o fluxo é direcionado para as respectivas atividades de entrada de dados, que culminam no cálculo e armazenamento de novas amostras. As opções '3', '4' e '5' conduzem a atividades de visualização de dados, como "Ver resumo das amostras" ou a geração de gráficos específicos através da invocação dos métodos correspondentes.

Caso a opção selecionada seja '6' ("Sair"), a atividade `m_running = false` é executada, o que garante a terminação do laço de repetição na sua próxima iteração. Qualquer outra entrada de usuário que não corresponda às opções válidas levará à atividade "Exibir 'Opção inválida!'", e o controle retornará ao início do laço, aguardando uma nova entrada.

Uma vez que a condição de saída do laço é satisfeita, o fluxo de controle avança para a atividade final "Chamar Finalize()", que executa os procedimentos de encerramento do sistema. O processo culmina, então, no Nó Final, representando a terminação completa e ordenada da aplicação.

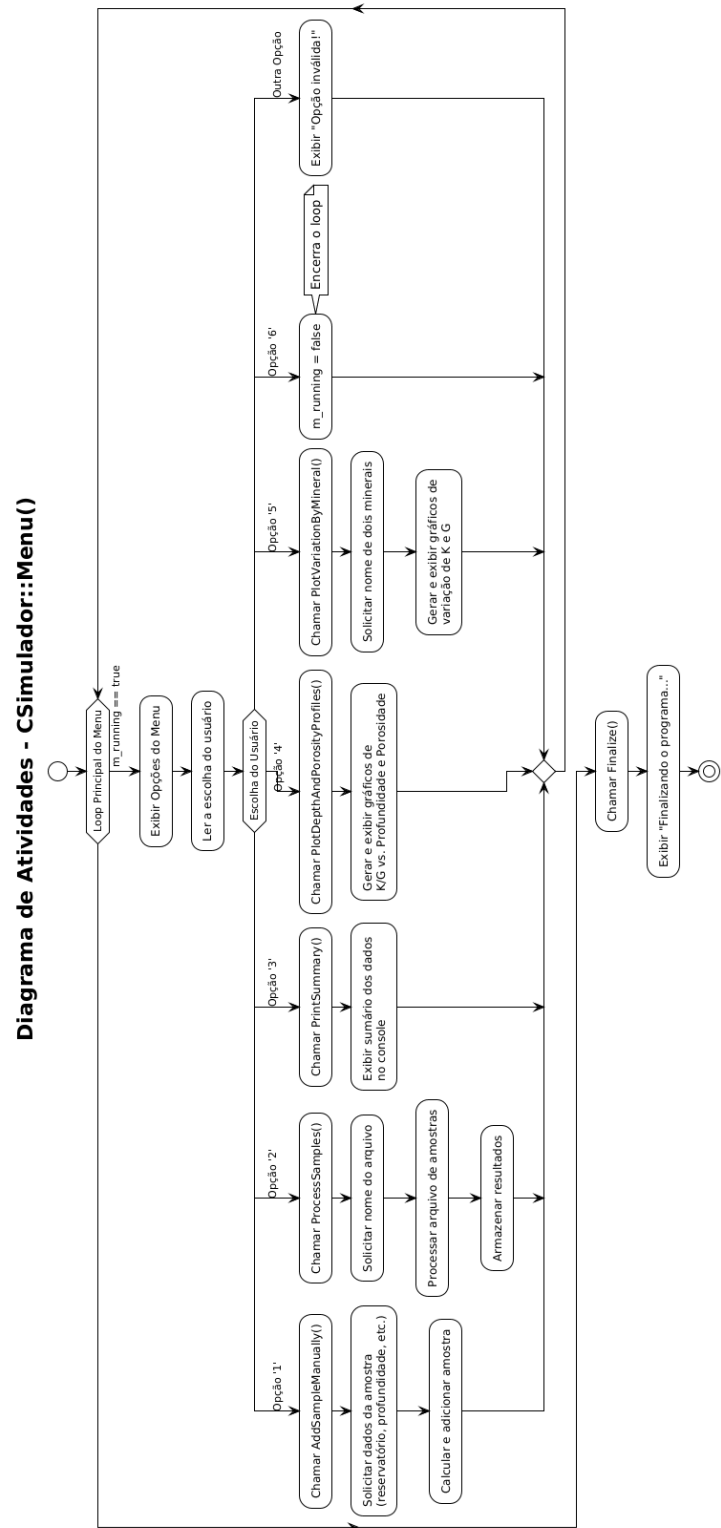


Figura 4.6: Diagrama de atividades

Capítulo 5

Projeto

Neste capítulo do projeto de engenharia veremos questões associadas ao projeto do sistema, incluindo protocolos, recursos, plataformas suportadas, implicações nos diagramas feitos anteriormente, diagramas de componentes e implantação. Na segunda parte revisamos os diagramas levando em conta as decisões do projeto do sistema.

5.1 Projeto do Sistema

Depois da análise orientada a objeto desenvolve-se o projeto do sistema, qual envolve etapas como a definição dos protocolos, da interface API, o uso de recursos, a subdivisão do sistema em subsistemas, a alocação dos subsistemas ao hardware e a seleção das estruturas de controle, a seleção das plataformas do sistema, das bibliotecas externas, dos padrões de projeto, além da tomada de decisões conceituais e políticas que formam a infraestrutura do projeto.

Deve-se definir padrões de documentação, padrões para o nome das classes, padrões de retorno e de parâmetros em métodos, características da interface do usuário e características de desempenho.

O projeto do sistema é a estratégia de alto nível para resolver o problema e elaborar uma solução.

1. Protocolos

- O sistema é concebido para ser uma aplicação desktop autônoma, com capacidade de importar dados e exportar resultados. A comunicação com dispositivos externos (como sensores laboratoriais, espectrômetros, etc.) não será necessária nesta primeira versão. No entanto, o software deverá:
 - Ler arquivos de entrada com composições mineralógicas em formatos abertos, preferencialmente .csv e .xlsx
 - Exportar os resultados dos cálculos em formatos igualmente abertos em .txt (relatórios simples); .csv (planilhas com resultados); .png (gráficos e visualizações)

- O sistema será implementado seguindo o paradigma de programação orientada a objetos, sendo os principais componentes do sistema representados como objetos com responsabilidades bem definidas.

A comunicação entre objetos seguirá os princípios do acoplamento fraco e alta coesão, utilizando:

- Chamadas diretas de métodos
 - Passagem de parâmetros por instância (composição e agregação)
 - Padrão de projeto MVC (Model-View-Controller), separando: Modelos físicos e matemáticos (Model); Interface com o usuário (View); Lógica de controle e fluxo de dados (Controller)
- O sistema utilizará os seguintes formatos abertos para arquivos de entrada:
 - .csv: para dados de composição mineralógica, módulos dos minerais, nomes de fases, etc.
 - .json: (opcional) para configurações de execução (modelo a usar, limites, parâmetros fixos)
 - O sistema utilizará os seguintes formatos abertos para arquivos de saída:
 - .csv: resultados tabulares
 - .txt: logs e relatórios legíveis
 - .png / .pdf: gráficos comparativos

2. Recursos

- O sistema utiliza um banco de dados embutido contendo as propriedades elásticas conhecidas dos minerais (módulo de compressibilidade K, módulo de cisalhamento G, profundidade da amostra, porosidade). Este banco de dados permite a automação da busca de propriedades físicas a partir da identificação do nome do mineral, eliminando a necessidade de inserção manual dos parâmetros. Exemplo de estrutura da tabela minerais:

nome	k	g
albita	55	29.5
anidrita	55	30
anortita	84	40
aragonita	47	38.5
⋮	⋮	⋮

3. Plataformas

- Padrão de Nomes
 - Classes: Utilizam o prefixo C seguido pelo formato PascalCase (ex: CInput, CSimulador).

- Métodos e Funções: Adotam o formato PascalCase (ex: ParseSampleBlock, GetBulkModulus).
- Variáveis Membro (Atributos): São prefixadas com m e seguem o formato camelCase (ex: mreservatorio, mdatabase).

4. Padrão de retorno e parâmetros

- Retorno de Dados: Os métodos que realizam cálculos ou processamentos retornam, preferencialmente, objetos que encapsulam os resultados, como instâncias de classes (CAmostra, Result) ou coleções padrão (std::vector).
- Tratamento de Erros: Em caso de falhas, como a não localização de um mineral na base de dados, o sistema adota a estratégia de notificar o erro através de mensagens no fluxo de erro padrão (std::cerr), permitindo que a execução prossiga de forma controlada. A implementação de um sistema de exceções customizadas é vislumbrada como um aprimoramento futuro para um tratamento de erros ainda mais robusto.

5.2 Projeto Orientado a Objeto – POO

O projeto orientado a objeto é a etapa posterior ao projeto do sistema. Baseia-se na análise, mas considera as decisões do projeto do sistema. Acrescenta à análise desenvolvida as características da plataforma escolhida (hardware, sistema operacional e linguagem de programação). Passa pelo maior detalhamento do funcionamento do software, acrescentando atributos e métodos que envolvem a solução de problemas específicos não identificados durante a análise.

Envolve a otimização da estrutura de dados e dos algoritmos, a minimização do tempo de execução, de memória e de custos. Existe um desvio de ênfase para os conceitos da plataforma selecionada.

Efeitos do projeto no modelo estrutural

- Neste projeto, o modelo estrutural foi enriquecido com a inclusão de componentes da Biblioteca Padrão (STL) do C++. Pacotes como <vector>, <string>, <map>, <fstream> e <sstream> foram fundamentais para a implementação das estruturas de dados e das operações de entrada/saída. Adicionalmente, a decisão de utilizar a ferramenta externa Gnuplot para a visualização de dados introduziu uma dependência de sistema, refletida na arquitetura através da classe CPlotManager, cuja responsabilidade é fazer a interface com este subsistema.
- A escolha da linguagem C++ e de suas bibliotecas padrão resultou na concretização de classes e associações. Por exemplo, a classe CInput utiliza std::ifstream para a leitura de arquivos, e a CPlotManager emprega std::ofstream para

gerar os scripts `.gp` e os arquivos de dados `.dat`. A classe `CAmostra` utiliza `std::map<std::string, float>` para materializar a associação entre a amostra e sua composição mineralógica, uma decisão de projeto que garante eficiência na busca por minerais.

- A plataforma escolhida impõe a dependência de um compilador C++11 (ou superior) para garantir a compatibilidade com os recursos da STL utilizados. A interação com o Gnuplot estabelece a restrição de que este software deve estar instalado e acessível no `PATH` do sistema operacional para que a funcionalidade de plotagem de gráficos opere corretamente.

Efeitos do projeto no modelo dinâmico

- A escolha da plataforma C++ e da biblioteca padrão influenciou diretamente os diagramas de sequência. Uma operação como "processar amostras" se desdobra em uma sequência detalhada: o objeto `CSimulador` invoca `CInput::ReadFile`, que internamente utiliza `std::ifstream` para ler o arquivo de entrada. Para cada bloco de amostra lido, `CSimulador` instancia `CAmostra` e invoca `CElasticCalculator::Calcula`. Este, por sua vez, interage com `CMineralDatabase` para obter as propriedades de cada mineral antes de retornar o resultado. Este nível de detalhe é uma consequência direta das decisões de projeto.
- A classe `CSimulador` implementa um laço de execução em seu método `Run()`, que opera com base em um menu interativo gerenciado pelo método `Menu()`. Este comportamento pode ser modelado por um diagrama de máquina de estados, onde cada opção do menu representa uma transição para um estado diferente da aplicação (ex: "Carregando Dados", "Calculando Propriedades", "Gerando Gráficos", "Finalizado"). A análise inicial poderia não ter previsto este nível de interação, que foi adicionado durante o projeto para melhorar a usabilidade da ferramenta.

Efeitos do projeto nos atributos

- Neste projeto, a transição da análise para o projeto introduziu atributos específicos da implementação. A classe `CMineralDatabase`, por exemplo, possui o atributo `m_databaseFilePath`, que armazena o caminho para o arquivo `.csv`, um detalhe de implementação para o acesso ao disco. Da mesma forma, as classes `CElasticCalculator`, `CBulkCalculator` e `CShearCalculator` contêm uma referência constante à base de dados (`m_db`), um ponteiro de implementação que otimiza o acesso aos dados dos minerais sem a necessidade de recarregá-los.

Efeitos do projeto nos métodos

- Os métodos da classe `CPlotManager` são um claro exemplo do impacto da plataforma escolhida. A necessidade de interagir com o Gnuplot levou à criação de métodos como `ExportCurve`, que realiza acesso a disco para criar arquivos `.dat`, e `GeneratePlotScript`, que não apenas escreve um script em um arquivo, mas também executa um comando de sistema (`system()`) para invocar um processo externo.
- No sistema, observa-se a distinção entre métodos políticos e de processamento. O método `CSimulador::Menu()` é eminentemente político, orquestrando o fluxo de controle da aplicação com base na entrada do usuário. Em contrapartida, os métodos `CalculateVoigt`, `CalculateReuss` e `CalculateHill` nas classes `CBulkCalculator` e `CShearCalculator` são puramente de processamento, focados na execução otimizada de fórmulas matemáticas.
- Os métodos implementados respondem diretamente às responsabilidades de suas classes. A responsabilidade da `CReservoirData` é armazenar e fornecer dados agregados. Seus métodos, como `GetKGVsDepth` e `GetKGVsPorosity`, cumprem precisamente esta função, preparando os dados no formato exato que a `CPlotManager` necessita.

Efeitos do projeto nas heranças

- Neste projeto, optou-se pela **composição em detrimento da herança**. Não há hierarquias de herança, uma decisão de projeto deliberada que favorece a flexibilidade e um acoplamento mais fraco. Por exemplo, a classe `CElasticCalculator` contém instâncias de `CBulkCalculator` e `CShearCalculator`, em vez de herdar delas. Esta abordagem simplifica o design, pois as responsabilidades de cálculo são delegadas a objetos especializados.

Efeitos do projeto nas associações

- As associações foram implementadas utilizando os contêineres da STL. A associação "um-para-muitos" entre `CReservoirData` e `CAmostra` é materializada por um `std::vector<Enriched Sample>`. Esta escolha permite o armazenamento ordenado e o acesso indexado às amostras processadas.
- A associação entre uma `CAmostra` e seus minerais é implementada com um `std::map<std::string,float>`. Este "dicionário" é ideal, pois usa o nome do mineral como chave para um acesso rápido (complexidade logarítmica) à sua proporção, além de garantir a unicidade.

Efeitos do projeto nas otimizações

- A principal otimização no design de dados é o uso de `std::map` na classe `CMineralDatabase` para armazenar os minerais. Isso garante que a busca por um mineral durante os cálculos seja realizada em tempo logarítmico, o que é significativamente mais eficiente do que uma busca linear.
- O sistema opera de forma sequencial. Uma futura otimização poderia envolver a utilização de concorrência (`std::thread` ou `std::async`) para processar múltiplas amostras em paralelo, otimizando o uso de processadores multi-core.
- O design plano e com baixo acoplamento, onde `CSimulador` mantém referências diretas aos subsistemas, já previne caminhos de acesso longos, tornando desnecessária a adição de associações extras para fins de otimização.

5.3 Diagrama de Componentes

Segue o diagrama de componentes:

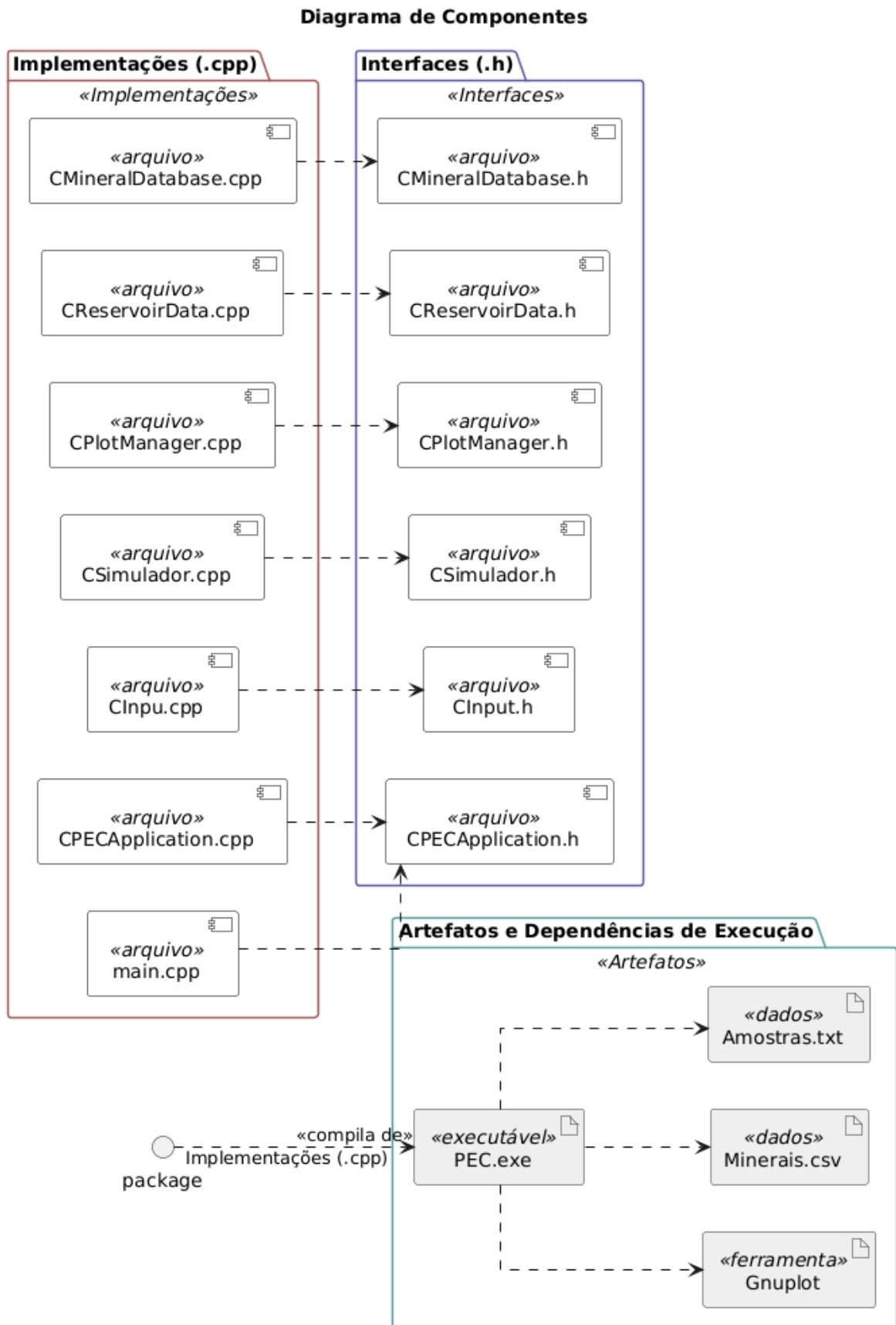


Figura 5.1: Diagrama de componentes

5.4 Diagrama de Implantação

O diagrama de implantação é um diagrama de alto nível que inclui relações entre o sistema e o hardware e que se preocupa com os aspectos da arquitetura computacional escolhida. Seu enfoque é o hardware, a configuração dos nós em tempo de execução.

O diagrama de implantação deve incluir os elementos necessários para que o sistema seja colocado em funcionamento: computador, periféricos, processadores, dispositivos, nós, relacionamentos de dependência, associação, componentes, subsistemas, restrições e notas.

Neste projeto, o sistema de cálculo poroelástico foi concebido para ser executado em um ambiente de workstation local, como um computador pessoal ou um servidor de menor porte, sem a necessidade de uma infraestrutura de cluster distribuído. A interação com o usuário ocorre através da interface de linha de comando, e a geração de gráficos depende da instalação local do Gnuplot.

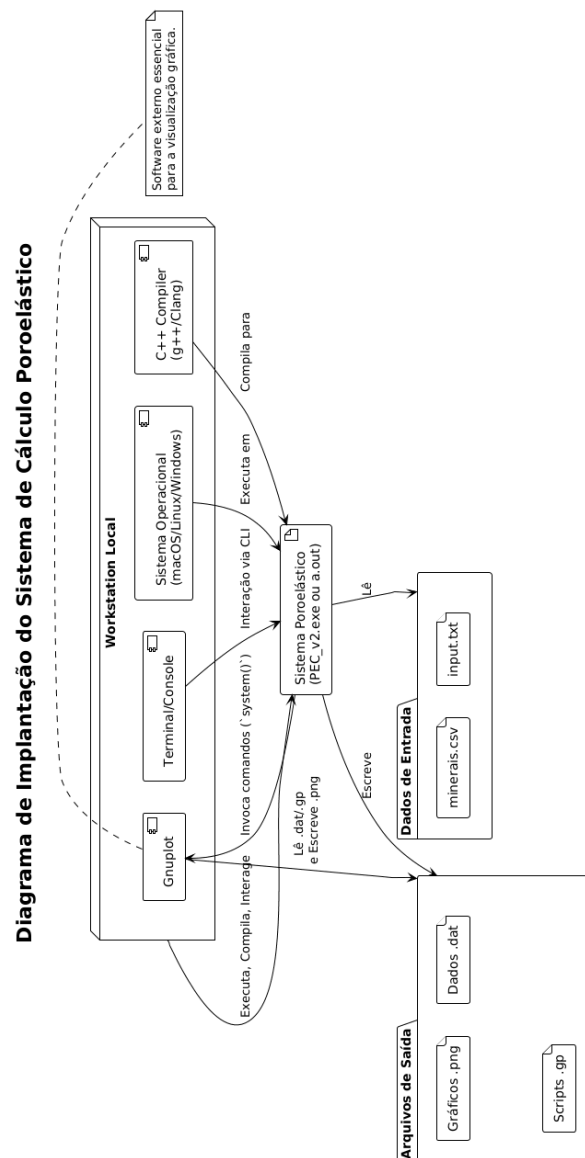


Figura 5.2: Diagrama de implantação

5.4.1 Lista de características «features»

Após a análise dos requisitos e considerando a natureza acadêmica e didática do projeto, a implementação foi dividida em duas versões principais, culminando na seguinte lista de funcionalidades:

Versão 1.0 - Núcleo de Cálculo Poroelástico

- **Cálculo dos Módulos Elásticos:** O sistema deve ser capaz de calcular os módulos de compressibilidade (K) e de cisalhamento (G) efetivos de um compósito mineral utilizando as médias de Voigt, Reuss e Hill.

Classes implementadas: CElasticCalculator, CBulkCalculator, CShearCalculator.

- **Gerenciamento da Base de Dados de Minerais:** O sistema deve carregar e gerenciar uma base de dados de minerais a partir de um arquivo externo (.csv), permitindo a consulta das propriedades elásticas de cada mineral pelo nome.

Classe implementada: CMineralDatabase.

- **Leitura de Arquivo de Entrada Simplificado:** O sistema deve ser capaz de ler um arquivo de texto com a definição de uma única amostra de rocha, incluindo sua composição mineralógica e porosidade.

Classe implementada: CInput.

- **Visualização Gráfica:** O sistema deve gerar gráficos que correlacionem as propriedades elásticas calculadas com a variação da composição entre dois minerais.

Classe implementada: CPlotManager.

- **Orquestração Central:** O sistema deve possuir uma classe principal para coordenar o fluxo de execução, desde a leitura dos dados até a apresentação dos resultados.

Classe implementada: CSimulador.

Versão 2.0 - Arquitetura Refinada e Análise Multi-Amostra

- **Abstração da Entidade Amostra:** Para aprimorar a coesão e o encapsulamento, o conceito de "amostra de rocha" foi abstraído em uma classe própria, responsável por conter todos os seus dados característicos.

Classe implementada: CAmostra.

- **Abstração da Entidade Mineral:** De forma análoga, a entidade "mineral" foi encapsulada em sua própria classe, refinando a estrutura da base de dados e tornando o código mais claro e orientado a objeto.

Classe implementada: CMineral.

- **Processamento de Múltiplas Amostras:** O sistema foi aprimorado para ler e processar arquivos de entrada contendo a definição de múltiplas amostras de diferentes profundidades e porosidades.

Evolução da classe: CInput.

- **Gerenciamento de Dados de Reservatório:** Foi introduzida uma classe para agregar e gerenciar o conjunto de todas as amostras processadas e seus respectivos resultados, facilitando a análise consolidada.

Classe implementada: CReservoirData.

- **Geração de Perfis de Profundidade e Porosidade:** A capacidade de visualização foi expandida para permitir a plotagem de perfis contínuos, mostrando a variação dos módulos elásticos (K e G) em função da profundidade ou da porosidade para o conjunto de amostras analisadas.

Evolução da classe: CPlotManager.

Capítulo 6

Ciclos de Planejamento/Detalhamento

Neste capítulo, lista-se as características do sistema desenvolvido, organizadas de forma incremental de acordo com as versões do projeto. Esta abordagem visa a demonstrar a evolução da arquitetura e da funcionalidade do software, desde sua concepção inicial até a versão final apresentada neste trabalho.

6.1 Lista de características «features»

Após a análise dos requisitos e considerando a natureza acadêmica e didática do projeto, a implementação foi dividida em duas versões principais, culminando na seguinte lista de funcionalidades:

Versão 1.0 - Núcleo de Cálculo Poroelástico

- **Cálculo dos Módulos Elásticos:** O sistema deve ser capaz de calcular os módulos de compressibilidade (K) e de cisalhamento (G) efetivos de um compósito mineral utilizando as médias de Voigt, Reuss e Hill.

Classes implementadas: CElasticCalculator, CBulkCalculator, CShearCalculator.

- **Gerenciamento da Base de Dados de Minerais:** O sistema deve carregar e gerenciar uma base de dados de minerais a partir de um arquivo externo (.csv), permitindo a consulta das propriedades elásticas de cada mineral pelo nome.

Classe implementada: CMineralDatabase.

- **Leitura de Arquivo de Entrada Simplificado:** O sistema deve ser capaz de ler um arquivo de texto com a definição de uma única amostra de rocha, incluindo sua composição mineralógica e porosidade.

Classe implementada: CInput.

- **Visualização Gráfica:** O sistema deve gerar gráficos que correlacionem as propriedades elásticas calculadas com a variação da composição entre dois minerais.

Classe implementada: CPlotManager.

- **Orquestração Central:** O sistema deve possuir uma classe principal para coordenar o fluxo de execução, desde a leitura dos dados até a apresentação dos resultados.

Classe implementada: CSimulador.

Versão 2.0 - Arquitetura Refinada e Análise Multi-Amostra

- **Abstração da Entidade Amostra:** Para aprimorar a coesão e o encapsulamento, o conceito de "amostra de rocha" foi abstraído em uma classe própria, responsável por conter todos os seus dados característicos.

Classe implementada: CAmostra.

- **Abstração da Entidade Mineral:** De forma análoga, a entidade "mineral" foi encapsulada em sua própria classe, refinando a estrutura da base de dados e tornando o código mais claro e orientado a objeto.

Classe implementada: CMineral.

- **Processamento de Múltiplas Amostras:** O sistema foi aprimorado para ler e processar arquivos de entrada contendo a definição de múltiplas amostras de diferentes profundidades e porosidades.

Evolução da classe: CInput.

- **Gerenciamento de Dados de Reservatório:** Foi introduzida uma classe para agregar e gerenciar o conjunto de todas as amostras processadas e seus respectivos resultados, facilitando a análise consolidada.

Classe implementada: CReservoirData.

- **Geração de Perfis de Profundidade e Porosidade:** A capacidade de visualização foi expandida para permitir a plotagem de perfis contínuos, mostrando a variação dos módulos elásticos (K e G) em função da profundidade ou da porosidade para o conjunto de amostras analisadas.

Evolução da classe: CPlotManager.

Capítulo 7

Ciclos Construção - Implementação

Este capítulo documenta integralmente o código-fonte desenvolvido para o software **PoreElasticCalculator (PEC v3)**. O projeto foi concebido sob a filosofia de *Open Source*, com o objetivo de democratizar o acesso a ferramentas de física de rochas no LENEP.

Abaixo estão os scripts de configuração e, na sequência, as definições de todas as interfaces do sistema (arquivos de cabeçalho).

7.1 Configuração e Scripts Auxiliares

```
1 cmake_minimum_required(VERSION 3.15)
2
3 # Nome do projeto e padrao C++
4 project(PoreElasticCalculator VERSION 3.0 LANGUAGES CXX)
5
6 # C++17 ou superior
7 set(CMAKE_CXX_STANDARD 17)
8 set(CMAKE_CXX_STANDARD_REQUIRED ON)
9
10 # Diretorios
11 include_directories(include)
12
13 # Lista de fontes
14 file(GLOB_RECURSE SOURCES
15     src/*.cpp
16     include/*.h
17     main.cpp
18 )
19
20 # Executavel principal
21 add_executable(${PROJECT_NAME} ${SOURCES})
22
23 # Opcoes para warnings
24 if (CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
```



```
25     target_compile_options(${PROJECT_NAME} PRIVATE -Wall -Wextra -  
        pedantic -Wno-unused-parameter)  
26 elseif (MSVC)  
27     target_compile_options(${PROJECT_NAME} PRIVATE /W4 /permissive-)  
28 endif()
```

Listing 7.1: CMakeLists.txt

```
1 #!/bin/bash  
2 cd "$(dirname "$0")/build"  
3 ./PoreElasticCalculator
```

Listing 7.2: Launch.command (macOS)

7.2 Interfaces do Sistema (.h)

A seguir, apresentam-se todos os arquivos de cabeçalho que definem a estrutura das classes e seus contratos.

7.2.1 Entidades Básicas

```
1 #ifndef CMINERAL_H  
2 #define CMINERAL_H  
3  
4 #include <string>  
5  
6 /**  
7  * @class CMineral  
8  * @brief Representa um mineral com modulo de bulk e cisalhamento.  
9  */  
10 class CMineral {  
11 public:  
12     /**  
13      * @brief Construtor padrao necessario para uso com std::  
        unordered_map.  
14      */  
15     CMineral();  
16  
17     /**  
18      * @brief Construtor com nome, bulk e shear.  
19      */  
20     CMineral(const std::string& nome, float bulk, float shear);  
21  
22     const std::string& Nome() const;  
23     float Bulk() const;  
24     float Shear() const;  
25  
26 private:
```

```

27     std::string m_nome;
28     float m_bulk;
29     float m_shear;
30 };
31
32 #endif

```

Listing 7.3: include/CMineral.h

```

1  #ifndef CAMOSTRA_H
2  #define CAMOSTRA_H
3
4  #include <string>
5  #include <map>
6
7  /**
8   * @class CAmostra
9   * @brief Representa os dados de uma amostra.
10  */
11  class CAmostra {
12  public:
13      CAmostra();
14      CAmostra(const std::string& reservatorio, const std::string& nome,
15              float profundidade, float porosidade,
16              const std::map<std::string, float>& minerais);
17
18      const std::string& NomeReservatorio() const;
19      const std::string& NomeAmostra() const;
20      float Profundidade() const;
21      float Porosidade() const;
22      const std::map<std::string, float>& Minerais() const;
23
24  private:
25      std::string m_reservatorio;
26      std::string m_nome;
27      float m_profundidade;
28      float m_porosidade;
29      std::map<std::string, float> m_minerais;
30  };
31
32 #endif

```

Listing 7.4: include/CAmostra.h

7.2.2 Gestão de Dados e Entrada

```

1  #ifndef CMINERALDATABASE_H
2  #define CMINERALDATABASE_H
3

```

```

4 #include "CMineral.h"
5 #include <unordered_map>
6 #include <string>
7 #include <vector>
8
9 /**
10  * @class CMineralDatabase
11  * @brief Banco de dados de minerais com seus modulos elasticos.
12  */
13 class CMineralDatabase {
14 public:
15     explicit CMineralDatabase(const std::string& filePath);
16
17     /**
18      * @brief Verifica se o mineral existe no banco.
19      */
20     bool HasMineral(const std::string& nome) const;
21
22     /**
23      * @brief Retorna o modulo de bulk do mineral.
24      */
25     float GetBulkModulus(const std::string& nome) const;
26
27     /**
28      * @brief Retorna o modulo de cisalhamento do mineral.
29      */
30     float GetShearModulus(const std::string& nome) const;
31
32     /**
33      * @brief Adiciona um novo mineral ao banco.
34      */
35     void AddMineral(const std::string& nome, float k, float g);
36
37     /**
38      * @brief Imprime todos os minerais ordenados alfabeticamente.
39      */
40     void PrintAllMineralsSorted() const;
41
42 private:
43     std::unordered_map<std::string, CMineral> m_minerals;
44
45     void LoadFromFile(const std::string& filePath);
46 };
47
48 #endif

```

Listing 7.5: include/CMineralDatabase.h

```

1 #ifndef CINPUTMANAGER_H
2 #define CINPUTMANAGER_H

```

```
3
4 #include "CAmostra.h"
5 #include <string>
6 #include <vector>
7 #include <map>
8
9 /**
10  * @class CInputManager
11  * @brief Gerencia a entrada de dados do usuario, seja via terminal ou
12  *        arquivo.
13  */
14 class CInput {
15 public:
16     CInput();
17     ~CInput();
18
19     /**
20      * @brief Solicita entrada manual do usuario via terminal.
21      * @return CAmostra preenchida.
22      */
23     CAmostra PromptManualInput();
24
25     /**
26      * @brief Le os dados de entrada a partir de um arquivo CSV.
27      * @param filePath Caminho para o arquivo CSV.
28      * @return CAmostra preenchida.
29      */
30     CAmostra ReadFromCSV(const std::string& filePath);
31
32     /**
33      * @brief Le os dados de entrada a partir de um arquivo de texto (
34      *        CSV ou TXT).
35      * @param filePath Caminho para o arquivo.
36      * @return CAmostra preenchida.
37      */
38     CAmostra ReadFromFile(const std::string& filePath);
39
40     /**
41      * @brief Imprime no terminal os dados lidos de uma amostra.
42      * @param amostra A amostra a ser impressa.
43      */
44     void PrintSampleInput(const CAmostra& amostra);
45
46     /**
47      * @brief Le multiplas amostras de um unico arquivo de entrada.
48      * @param filePath Caminho para o arquivo.
49      * @return Vetor de amostras lidas.
50      */
51     std::vector<CAmostra> ReadMultipleFromFile(const std::string&
```

```

    filePath);
50
51 private:
52     /**
53      * @brief Funcao auxiliar para fazer parsing de uma linha CSV.
54      */
55     void ParseCSVLine(const std::string& line, std::map<std::string,
float>& mineralsOut);
56 };
57
58 #endif

```

Listing 7.6: include/CInput.h

```

1 #ifndef CRESERVOIRDATA_H
2 #define CRESERVOIRDATA_H
3
4 #include "CAmostra.h"
5 #include "CElasticCalculator.h"
6 #include <vector>
7 #include <set>
8
9 /**
10  * @class CReservoirData
11  * @brief Armazena todas as amostras e seus resultados elasticos.
12  */
13 class CReservoirData {
14 public:
15     std::set<std::string> ObterNomesDeMineraisUnicos() const;
16     struct EnrichedSample {
17         CAmostra amostra;
18         CElasticCalculator::Result resultado;
19
20     };
21
22     void AddSample(const CAmostra& amostra, const CElasticCalculator::
Result& resultado);
23     const std::vector<EnrichedSample>& GetSamples() const;
24
25     void PrintSummary() const;
26
27     void GetKGVsDepth(std::vector<float>& depth,
28                     std::vector<float>& bulkHill,
29                     std::vector<float>& shearHill) const;
30
31     void GetKGVsPorosity(std::vector<float>& phi,
32                         std::vector<float>& bulkHill,
33                         std::vector<float>& shearHill) const;
34
35     void GetKGVsDepthFull(std::vector<float>& depth,

```

```

36         std::vector<float>& kV, std::vector<float>&
kR, std::vector<float>& kH,
37         std::vector<float>& gV, std::vector<float>&
gR, std::vector<float>& gH) const;
38
39     void GetKGVsPorosityFull(std::vector<float>& phi,
40                             std::vector<float>& kV, std::vector<float>
>& kR, std::vector<float>& kH,
41                             std::vector<float>& gV, std::vector<float>
>& gR, std::vector<float>& gH) const;
42
43 private:
44     std::vector<EnrichedSample> m_amostras;
45 };
46
47 #endif // CRESERVOIRDATA_H

```

Listing 7.7: include/CReservoirData.h

7.2.3 Motor de Cálculo (Física)

```

1  #ifndef CBULKCALCULATOR_H
2  #define CBULKCALCULATOR_H
3
4  #include "CMineralDatabase.h"
5  #include <map>
6  #include <string>
7
8  /**
9   * @class CBulkCalculator
10  * @brief Calcula os modulos de bulk usando os teoremas de Voigt e
    Reuss.
11  */
12 class CBulkCalculator {
13 public:
14     explicit CBulkCalculator(const CMineralDatabase& db);
15
16     float CalcVoigt(const std::map<std::string, float>& composicao)
const;
17     float CalcReuss(const std::map<std::string, float>& composicao)
const;
18
19 private:
20     const CMineralDatabase& m_db;
21 };
22
23 #endif

```

Listing 7.8: include/CBulkCalculator.h

```

1 #ifndef CSHEARCALCULATOR_H
2 #define CSHEARCALCULATOR_H
3
4 #include "CMineralDatabase.h"
5 #include <map>
6 #include <string>
7
8 /**
9  * @class CShearCalculator
10  * @brief Calcula os modulos de cisalhamento usando os teoremas de
11  *        Voigt e Reuss.
12  */
13 class CShearCalculator {
14 public:
15     explicit CShearCalculator(const CMineralDatabase& db);
16
17     float CalcVoigt(const std::map<std::string, float>& composicao)
18     const;
19     float CalcReuss(const std::map<std::string, float>& composicao)
20     const;
21 private:
22     const CMineralDatabase& m_db;
23 };
24 #endif // CSHEARCALCULATOR_H

```

Listing 7.9: include/CShearCalculator.h

```

1 #ifndef CELASTICCALCULATOR_H
2 #define CELASTICCALCULATOR_H
3
4 #include "CMineralDatabase.h"
5 #include "CAmostra.h"
6 #include "CBulkCalculator.h"
7 #include "CShearCalculator.h"
8
9 /**
10  * @class CElasticCalculator
11  * @brief Calcula os modulos elasticos da amostra usando as medias de
12  *        Voigt, Reuss e Hill.
13  */
14 class CElasticCalculator {
15 public:
16     struct Result {
17         float bulkVoigt, bulkReuss, bulkHill;
18         float shearVoigt, shearReuss, shearHill;
19     };
20
21     explicit CElasticCalculator(const CMineralDatabase& db);

```

```

21
22     /**
23     * @brief Realiza o calculo completo dos modulos para uma amostra.
24     * @param amostra A amostra de entrada.
25     * @return Resultado com todos os valores de K e G.
26     */
27     Result Calculate(const CAmostra& amostra) const;
28
29 private:
30     CBulkCalculator m_bulkCalc;
31     CShearCalculator m_shearCalc;
32 };
33
34 #endif // CELASTICCALCULATOR_H

```

Listing 7.10: include/CElasticCalculator.h

7.2.4 Visualização e Controle

```

1 #ifndef CPLOTMANAGER_H
2 #define CPLOTMANAGER_H
3
4 #include "CReservoirData.h"
5 #include "CMineralDatabase.h"
6 #include <string>
7 #include <vector>
8
9 /**
10 * @class CPlotManager
11 * @brief Gerencia a criacao e exibicao/salvamento de graficos usando
12 * Gnuplot.
13 */
14 class CPlotManager {
15 public:
16     CPlotManager();
17
18     /**
19     * @brief Gera os graficos de K e G versus Porosidade.
20     * @param data Objeto contendo os dados das amostras.
21     * @param displayOnScreen Se verdadeiro, exibe o grafico em uma
22     * janela interativa.
23     */
24     void PlotPorosityProfile(const CReservoirData& data, bool
25     displayOnScreen);
26
27     /**
28     * @brief Gera os graficos de K e G versus Profundidade.
29     * @param data Objeto contendo os dados das amostras.
30     */
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```



```

27     * @param displayOnScreen Se verdadeiro, exibe o grafico em uma
janela interativa.
28     */
29     void PlotDepthProfile(const CReservoirData& data, bool
displayOnScreen);
30
31     /**
32     * @brief Gera os graficos de variacao mineralogica entre dois
minerais.
33     * @param db A base de dados de minerais.
34     * @param principal O nome do mineral principal (fixo).
35     * @param secundario O nome do mineral cuja proporcao ira variar.
36     * @param displayOnScreen Se verdadeiro, exibe o grafico em uma
janela interativa.
37     */
38     void PlotVariationByMineral(const CMineralDatabase& db,
39                                const std::string& principal,
40                                const std::string& secundario,
41                                bool displayOnScreen);
42
43 private:
44     std::string m_outputPath;
45
46     /**
47     * @brief Gera arquivos .dat com dados (x, y) no diretorio de
saida.
48     */
49     void ExportCurve(const std::string& filename, const std::vector<
double>& x, const std::vector<double>& y) const;
50
51     /**
52     * @brief Gera script Gnuplot, salva o grafico em PNG no diretorio
de saida e opcionalmente exibe na tela.
53     */
54     void GeneratePlotScript(const std::string& filenameGp,
55                             const std::string& outputPng,
56                             const std::string& title,
57                             const std::string& xlabel,
58                             const std::string& ylabel,
59                             const std::vector<std::string>& datFiles,
60                             const std::vector<std::string>& legends,
61                             bool displayOnScreen) const;
62 };
63
64 #endif

```

Listing 7.11: include/CPlotManager.h

```

1 #ifndef CPECAPPLICATION_H
2 #define CPECAPPLICATION_H

```

```
3
4 #include "CInput.h"
5 #include "CElasticCalculator.h"
6 #include "CMineralDatabase.h"
7 #include "CReservoirData.h"
8 #include "CPlotManager.h"
9
10 /**
11  * @class CSimulador
12  * @brief Orquestra a execucao do sistema de calculo poroelastico.
13  */
14 class CSimulador {
15 public:
16     CSimulador();
17     void Run();
18
19 private:
20     CInput m_inputManager;
21     CMineralDatabase m_database;
22     CElasticCalculator m_calculator;
23     CReservoirData m_reservoirData;
24     CPlotManager m_plotManager;
25     bool m_running;
26     bool m_displayGraphsOnScreen;
27
28     void Menu();
29     void Initialize();
30     void Finalize();
31     void ConfigureGraphDisplay();
32 };
33
34 #endif
```

Listing 7.12: include/CSimulador.h

7.3 Implementação (.cpp)

Esta seção apresenta a implementação concreta da lógica do sistema, organizada por módulos funcionais.

7.3.1 Entidades do Domínio

```
1 #include "CMineral.h"
2
3 CMineral::CMineral() : m_nome(""), m_bulk(0.0f), m_shear(0.0f) {}
4
5 CMineral::CMineral(const std::string& nome, float bulk, float shear)
```

```
6      : m_nome(nome), m_bulk(bulk), m_shear(shear) {}
7
8  const std::string& CMineral::Nome() const {
9      return m_nome;
10 }
11
12 float CMineral::Bulk() const {
13     return m_bulk;
14 }
15
16 float CMineral::Shear() const {
17     return m_shear;
18 }
```

Listing 7.13: src/CMineral.cpp

```
1 #include "CAmostra.h"
2
3 CAmostra::CAmostra()
4     : m_profundidade(0.0f), m_porosidade(0.0f) {}
5
6 CAmostra::CAmostra(const std::string& reservatorio, const std::string&
7     nome,
8                     float profundidade, float porosidade,
9                     const std::map<std::string, float>& minerais)
10     : m_reservatorio(reservatorio), m_nome(nome),
11       m_profundidade(profundidade), m_porosidade(porosidade),
12       m_minerais(minerais) {}
13
14 const std::string& CAmostra::NomeReservatorio() const {
15     return m_reservatorio;
16 }
17
18 const std::string& CAmostra::NomeAmostra() const {
19     return m_nome;
20 }
21
22 float CAmostra::Profundidade() const {
23     return m_profundidade;
24 }
25
26 float CAmostra::Porosidade() const {
27     return m_porosidade;
28 }
29
30 const std::map<std::string, float>& CAmostra::Minerais() const {
31     return m_minerais;
32 }
```

Listing 7.14: src/CAmostra.cpp

7.3.2 Gestão de Dados e Entrada (I/O)

```

1 #include "CMineralDatabase.h"
2 #include <fstream>
3 #include <iostream>
4 #include <sstream>
5 #include <algorithm>
6
7 CMineralDatabase::CMineralDatabase(const std::string& filePath) {
8     LoadFromFile(filePath);
9 }
10
11 void CMineralDatabase::LoadFromFile(const std::string& filePath) {
12     std::ifstream file(filePath);
13     if (!file.is_open()) {
14         std::cerr << "[ERRO] Nao foi possivel abrir o arquivo de
15         minerais: " << filePath << "\n";
16         return;
17     }
18     std::string linha;
19     bool headerPulado = false;
20     while (std::getline(file, linha)) {
21         if (!headerPulado) {
22             headerPulado = true;
23             continue;
24         }
25
26         std::stringstream ss(linha);
27         std::string nome, kStr, gStr;
28
29         if (std::getline(ss, nome, ',') &&
30             std::getline(ss, kStr, ',') &&
31             std::getline(ss, gStr)) {
32             try {
33                 float k = std::stof(kStr);
34                 float g = std::stof(gStr);
35                 AddMineral(nome, k, g);
36             } catch (...) {
37                 std::cerr << "[WARN] Erro ao interpretar linha: " <<
38                 linha << "\n";
39             }
40         }
41     }
42     file.close();
43 }
44
45 bool CMineralDatabase::HasMineral(const std::string& nome) const {

```

```

46     return m_minerals.count(nome) > 0;
47 }
48
49 float CMineralDatabase::GetBulkModulus(const std::string& nome) const
50 {
51     return m_minerals.at(nome).Bulk();
52 }
53 float CMineralDatabase::GetShearModulus(const std::string& nome) const
54 {
55     return m_minerals.at(nome).Shear();
56 }
57 void CMineralDatabase::AddMineral(const std::string& nome, float k,
58     float g) {
59     m_minerals[nome] = CMineral(nome, k, g);
60 }
61 void CMineralDatabase::PrintAllMineralsSorted() const {
62     std::vector<std::string> nomes;
63     for (const auto& [nome, _] : m_minerals)
64         nomes.push_back(nome);
65
66     std::sort(nomes.begin(), nomes.end());
67
68     for (const auto& nome : nomes) {
69         const auto& m = m_minerals.at(nome);
70         std::cout << nome << " => K: " << m.Bulk() << ", G: " << m.
71         Shear() << "\n";
72     }
73 }

```

Listing 7.15: src/CMineralDatabase.cpp

```

1  #include "CInput.h"
2  #include <iostream>
3  #include <fstream>
4  #include <sstream>
5  #include <algorithm>
6
7  CInput::CInput() {}
8  CInput::~CInput() {}
9
10 CAmostra CInput::PromptManualInput() {
11     std::string reservatorio, nome;
12     float profundidade, porosidade;
13     std::map<std::string, float> minerais;
14
15     std::cout << "Nome do reservatorio: ";
16     std::getline(std::cin, reservatorio);

```

```
17     std::cout << "Nome da amostra: ";
18     std::getline(std::cin, nome);
19     std::cout << "Profundidade (m): ";
20     std::cin >> profundidade;
21     std::cout << "Porosidade (%): ";
22     std::cin >> porosidade;
23     std::cin.ignore();
24
25     std::cout << "Digite os minerais e suas porcentagens (ex: quartzo
26     50 albita 50):\n";
27     std::string linha;
28     std::getline(std::cin, linha);
29     std::istringstream iss(linha);
30     std::string mineral;
31     float frac;
32     while (iss >> mineral >> frac)
33         minerais[mineral] = frac;
34
35     return CAmostra(reservatorio, nome, profundidade, porosidade,
36     minerais);
37 }
38
39 CAmostra CInput::ReadFromCSV(const std::string& filePath) {
40     std::ifstream file(filePath);
41     std::string linha;
42
43     std::getline(file, linha);
44     std::getline(file, linha);
45
46     std::stringstream ss(linha);
47     std::string reservatorio, nome;
48     float profundidade, porosidade;
49     std::map<std::string, float> minerais;
50
51     std::getline(ss, reservatorio, ',');
52     std::getline(ss, nome, ',');
53     ss >> profundidade;
54     ss.ignore(1);
55     ss >> porosidade;
56     ss.ignore(1);
57
58     std::string mineraisStr;
59     std::getline(ss, mineraisStr);
60     ParseCSVLine(mineraisStr, minerais);
61
62     return CAmostra(reservatorio, nome, profundidade, porosidade,
63     minerais);
64 }
```

```

63 CAmostra CInput::ReadFromFile(const std::string& filePath) {
64     std::ifstream file(filePath);
65     std::string linha;
66
67     std::string reservatorio, nome;
68     float profundidade = 0.0f, porosidade = 0.0f;
69     std::map<std::string, float> minerais;
70
71     while (std::getline(file, linha)) {
72         if (linha.empty()) break;
73         std::istringstream iss(linha);
74         std::string token;
75         iss >> token;
76
77         if (token == "reservatorio:" || token == "reservat rio:")
78             iss >> reservatorio;
79         else if (token == "amostra:")
80             iss >> nome;
81         else if (token == "profundidade:")
82             iss >> profundidade;
83         else if (token == "porosidade:")
84             iss >> porosidade;
85         else {
86             std::string mineral = token;
87             float perc;
88             iss >> perc;
89             minerais[mineral] = perc;
90         }
91     }
92
93     return CAmostra(reservatorio, nome, profundidade, porosidade,
94                     minerais);
95 }
96
97 void CInput::PrintSampleInput(const CAmostra& amostra) {
98     std::cout << "\n=== Dados da Amostra ===\n";
99     std::cout << "Reservatorio: " << amostra.NomeReservatorio() << "\n";
100     std::cout << "Amostra: " << amostra.NomeAmostra() << "\n";
101     std::cout << "Profundidade: " << amostra.Profundidade() << " m\n";
102     std::cout << "Porosidade: " << amostra.Porosidade() << " %\n";
103     std::cout << "\nComposicao Mineralogica:\n";
104     for (const auto& [nome, perc] : amostra.Minerais())
105         std::cout << " - " << nome << ": " << perc << "%\n";
106     std::cout << "=====\n\n";
107 }
108
109 std::vector<CAmostra> CInput::ReadMultipleFromFile(const std::string&
110 filePath) {

```

```
109     std::ifstream file(filePath);
110     std::vector<CAmostra> amostras;
111
112     std::string linha;
113     std::string nomeReservatorio, nomeAmostra;
114     float profundidade = 0.0f, porosidade = 0.0f;
115     std::map<std::string, float> minerais;
116
117     auto salvarAmostraSeValida = [&]() {
118         if (!nomeAmostra.empty() && profundidade > 0.0f && porosidade
119             >= 0.0f && !minerais.empty()) {
120             amostras.emplace_back(nomeReservatorio, nomeAmostra,
121                 profundidade, porosidade, minerais);
122             nomeAmostra.clear();
123             profundidade = 0.0f;
124             porosidade = 0.0f;
125             minerais.clear();
126         };
127
128         while (std::getline(file, linha)) {
129             if (linha.empty()) continue;
130
131             std::istringstream iss(linha);
132             std::string token;
133             iss >> token;
134
135             if (token == "reservatorio:" || token == "reservat rio:") {
136                 salvarAmostraSeValida();
137                 iss >> nomeReservatorio;
138             }
139             else if (token == "amostra:") {
140                 iss >> nomeAmostra;
141             }
142             else if (token == "profundidade:") {
143                 iss >> profundidade;
144             }
145             else if (token == "porosidade:") {
146                 iss >> porosidade;
147             }
148             else {
149                 std::string mineral = token;
150                 float perc;
151                 iss >> perc;
152                 minerais[mineral] = perc;
153             }
154         }
155
156         salvarAmostraSeValida();
157     }
```



```

156     return amostras;
157 }
158
159 void CInput::ParseCSVLine(const std::string& line, std::map<std::
    string, float>& mineralsOut) {
160     std::stringstream ss(line);
161     std::string token;
162
163     while (std::getline(ss, token, ',')) {
164         std::string mineral = token;
165         float perc = 0.0f;
166
167         if (std::getline(ss, token, ','))
168             perc = std::stof(token);
169
170         mineralsOut[mineral] = perc;
171     }
172 }

```

Listing 7.16: src/CInput.cpp

```

1  #include "CReservoirData.h"
2  #include <iostream>
3  #include <iomanip>
4
5  void CReservoirData::AddSample(const CAmostra& amostra, const
    CElasticCalculator::Result& resultado) {
6      m_amostras.push_back({amostra, resultado});
7  }
8
9  const std::vector<CReservoirData::EnrichedSample>& CReservoirData::
    GetSamples() const {
10     return m_amostras;
11 }
12
13 void CReservoirData::PrintSummary() const {
14     if (m_amostras.empty()) {
15         std::cout << "\n[INFO] Nenhuma amostra para exibir.\n";
16         return;
17     }
18
19     const int LARGURA_NOME = 12;
20     const int LARGURA_PROF = 18;
21     const int LARGURA_PORO = 16;
22     const int LARGURA_K = 29;
23     const int LARGURA_G = 24;
24
25     std::cout << "\n" << std::left
26         << std::setw(LARGURA_NOME) << "Nome" << "|" <<
27         << std::setw(LARGURA_PROF) << "Profundidade (m)" << "|" <<

```

```

28         << std::setw(LARGURA_PORO) << "Porosidade (%)" << "| "
29         << std::setw(LARGURA_K) << "Compressibilidade K (GPa)"
        << "| "
30         << std::setw(LARGURA_G) << "Cisalhamento G (GPa)" << std
        ::endl;
31
32     std::cout << std::string(LARGURA_NOME, '-') << "|-"
33     << std::string(LARGURA_PROF, '-') << "|-"
34     << std::string(LARGURA_PORO, '-') << "|-"
35     << std::string(LARGURA_K, '-') << "|-"
36     << std::string(LARGURA_G, '-') << std::endl;
37
38     std::cout << std::fixed << std::setprecision(2);
39
40     for (const auto& s : m_amostras) {
41         std::cout << std::left
42             << std::setw(LARGURA_NOME) << s.amostra.NomeAmostra
        () << "| "
43             << std::setw(LARGURA_PROF) << s.amostra.Profundidade
        () << "| "
44             << std::setw(LARGURA_PORO) << s.amostra.Porosidade()
        << "| "
45             << std::setw(LARGURA_K) << s.resultado.bulkHill << "
        | "
46             << std::setw(LARGURA_G) << s.resultado.shearHill <<
        std::endl;
47     }
48 }
49
50 void CReservoirData::GetKGVsDepth(std::vector<float>& depth,
51                                     std::vector<float>& bulkHill,
52                                     std::vector<float>& shearHill) const
53 {
54     depth.clear(); bulkHill.clear(); shearHill.clear();
55     for (const auto& s : m_amostras) {
56         depth.push_back(s.amostra.Profundidade());
57         bulkHill.push_back(s.resultado.bulkHill);
58         shearHill.push_back(s.resultado.shearHill);
59     }
60
61     void CReservoirData::GetKGVsPorosity(std::vector<float>& phi,
62                                             std::vector<float>& bulkHill,
63                                             std::vector<float>& shearHill)
64     const {
65         phi.clear(); bulkHill.clear(); shearHill.clear();
66         for (const auto& s : m_amostras) {
67             phi.push_back(s.amostra.Porosidade());
68             bulkHill.push_back(s.resultado.bulkHill);

```

```

68         shearHill.push_back(s.resultado.shearHill);
69     }
70 }
71
72 void CReservoirData::GetKGVsDepthFull(std::vector<float>& depth,
73                                       std::vector<float>& kV, std:::
74                                       vector<float>& kR, std:::vector<float>& kH,
75                                       std:::vector<float>& gV, std:::
76                                       vector<float>& gR, std:::vector<float>& gH) const {
77     depth.clear(); kV.clear(); kR.clear(); kH.clear(); gV.clear(); gR.
78     clear(); gH.clear();
79     for (const auto& s : m_amostras) {
80         depth.push_back(s.amostra.Profundidade());
81         kV.push_back(s.resultado.bulkVoigt);
82         kR.push_back(s.resultado.bulkReuss);
83         kH.push_back(s.resultado.bulkHill);
84         gV.push_back(s.resultado.shearVoigt);
85         gR.push_back(s.resultado.shearReuss);
86         gH.push_back(s.resultado.shearHill);
87     }
88 }
89
90 void CReservoirData::GetKGVsPorosityFull(std::vector<float>& phi,
91                                           std:::vector<float>& kV, std:::
92                                           vector<float>& kR, std:::vector<float>& kH,
93                                           std:::vector<float>& gV, std:::
94                                           vector<float>& gR, std:::vector<float>& gH) const {
95     phi.clear(); kV.clear(); kR.clear(); kH.clear(); gV.clear(); gR.
96     clear(); gH.clear();
97     for (const auto& s : m_amostras) {
98         phi.push_back(s.amostra.Porosidade());
99         kV.push_back(s.resultado.bulkVoigt);
100        kR.push_back(s.resultado.bulkReuss);
101        kH.push_back(s.resultado.bulkHill);
102        gV.push_back(s.resultado.shearVoigt);
103        gR.push_back(s.resultado.shearReuss);
104        gH.push_back(s.resultado.shearHill);
105    }
106 }
107
108 std::set<std::string> CReservoirData::ObterNomesDeMineraisUnicos()
109 const {
110     std::set<std::string> nomesUnicos;
111     for (const auto& s : m_amostras) {
112         auto minerais = s.amostra.Minerais();
113         for (const auto& parMineral : minerais) {
114             nomesUnicos.insert(parMineral.first);
115         }
116     }
117 }

```

```

110     return nomesUnicos;
111 }

```

Listing 7.17: src/CReservoirData.cpp

7.3.3 Motor de Cálculo (Física de Rochas)

```

1 #include "CBulkCalculator.h"
2
3 CBulkCalculator::CBulkCalculator(const CMineralDatabase& db)
4     : m_db(db) {}
5
6 float CBulkCalculator::CalcVoigt(const std::map<std::string, float>&
7     composicao) const {
8     float soma = 0.0f;
9     for (const auto& [nome, frac] : composicao)
10         soma += (frac / 100.0f) * m_db.GetBulkModulus(nome);
11     return soma;
12 }
13
14 float CBulkCalculator::CalcReuss(const std::map<std::string, float>&
15     composicao) const {
16     float inv = 0.0f;
17     for (const auto& [nome, frac] : composicao)
18         inv += (frac / 100.0f) / m_db.GetBulkModulus(nome);
19     return (inv > 0.0f) ? 1.0f / inv : -1.0f;
20 }

```

Listing 7.18: src/CBulkCalculator.cpp

```

1 #include "CShearCalculator.h"
2
3 CShearCalculator::CShearCalculator(const CMineralDatabase& db)
4     : m_db(db) {}
5
6 float CShearCalculator::CalcVoigt(const std::map<std::string, float>&
7     composicao) const {
8     float soma = 0.0f;
9     for (const auto& [nome, frac] : composicao)
10         soma += (frac / 100.0f) * m_db.GetShearModulus(nome);
11     return soma;
12 }
13
14 float CShearCalculator::CalcReuss(const std::map<std::string, float>&
15     composicao) const {
16     float inv = 0.0f;
17     for (const auto& [nome, frac] : composicao)
18         inv += (frac / 100.0f) / m_db.GetShearModulus(nome);
19     return (inv > 0.0f) ? 1.0f / inv : -1.0f;
20 }

```

18 }

Listing 7.19: src/CShearCalculator.cpp

```

1 #include "CElasticCalculator.h"
2
3 CElasticCalculator::CElasticCalculator(const CMineralDatabase& db)
4     : m_bulkCalc(db), m_shearCalc(db) {}
5
6 CElasticCalculator::Result CElasticCalculator::Calculate(const
7     CAmostra& amostra) const {
8     Result r;
9     const auto& composicao = amostra.Minerais();
10
11     r.bulkVoigt = m_bulkCalc.CalcVoigt(composicao);
12     r.bulkReuss = m_bulkCalc.CalcReuss(composicao);
13     r.bulkHill = (r.bulkVoigt + r.bulkReuss) / 2.0f;
14
15     r.shearVoigt = m_shearCalc.CalcVoigt(composicao);
16     r.shearReuss = m_shearCalc.CalcReuss(composicao);
17     r.shearHill = (r.shearVoigt + r.shearReuss) / 2.0f;
18
19     return r;
20 }

```

Listing 7.20: src/CElasticCalculator.cpp

7.3.4 Visualização e Orquestração

```

1 #include "CPlotManager.h"
2 #include <fstream>
3 #include <sstream>
4 #include <iomanip>
5 #include <cstdlib>
6 #include <cstdio>
7 #include <iostream>
8 #include <vector>
9 #include <filesystem>
10 #include <chrono>
11 #include <ctime>
12
13 static std::string getCurrentTimestamp() {
14     auto now = std::chrono::system_clock::now();
15     auto in_time_t = std::chrono::system_clock::to_time_t(now);
16     std::stringstream ss;
17     ss << std::put_time(std::localtime(&in_time_t), "%Y%m%d_%H%M%S");
18     return ss.str();
19 }
20

```

```

21 CPlotManager::CPlotManager() {
22     m_outputPath = "../test/plots/";
23     try {
24         if (!std::filesystem::exists(m_outputPath)) {
25             std::filesystem::create_directories(m_outputPath);
26         }
27     } catch (const std::exception& e) {
28         std::cerr << "Erro ao criar diretorio de plots: " << e.what()
29         << std::endl;
30     }
31 }
32 void CPlotManager::ExportCurve(const std::string& filename, const std
33 ::vector<double>& x, const std::vector<double>& y) const {
34     std::ofstream file(m_outputPath + filename);
35     for (size_t i = 0; i < x.size(); ++i)
36         file << std::fixed << std::setprecision(6) << x[i] << " " << y
37         [i] << "\n";
38 }
39 void CPlotManager::GeneratePlotScript(const std::string& filenameGp,
40                                     const std::string& outputPng,
41                                     const std::string& title,
42                                     const std::string& xlabel,
43                                     const std::string& ylabel,
44                                     const std::vector<std::string>&
45                                     datFiles,
46                                     const std::vector<std::string>&
47                                     legends,
48                                     bool displayOnScreen) const {
49     std::vector<std::string> estilos = {
50         "with lines lt 1 lw 2 dashtype 2",
51         "with lines lt 2 lw 2 dashtype 3",
52         "with lines lt 3 lw 2 dashtype 1"
53     };
54     std::ofstream gp(m_outputPath + filenameGp);
55     gp << "set terminal pngcairo size 800,600 enhanced font 'Arial
56     ,10'\n";
57     gp << "set output '" << m_outputPath + outputPng << "'\n";
58     gp << "set xlabel '" << xlabel << "'\n";
59     gp << "set ylabel '" << ylabel << "'\n";
60     gp << "set title '" << title << "'\n";
61     gp << "set grid\n";
62     gp << "plot \\n";
63     for (size_t i = 0; i < datFiles.size(); ++i) {
64         gp << "' " << m_outputPath + datFiles[i] << "' using 1:2 title
65         ' " << legends[i] << "' " << estilos[i];

```

```

63         if (i < datFiles.size() - 1) gp << ", \\n";
64         else gp << "\\n";
65     }
66     gp << "set output\\n";
67
68     if (displayOnScreen) {
69         gp << "set terminal wxt size 800,600\\n";
70         gp << "replot\\n";
71         gp << "pause -1 'Pressione Enter no terminal para fechar o
grafico'\\n";
72     }
73     gp.close();
74
75     std::string cmd = "gnuplot \"" + m_outputPath + filenameGp + "\"";
76
77     system(cmd.c_str());
78
79     std::remove((m_outputPath + filenameGp).c_str());
80     for (const auto& f : datFiles) std::remove((m_outputPath + f).
c_str());
81 }
82
83 void CPlotManager::PlotPorosityProfile(const CReservoirData& data,
bool displayOnScreen) {
84     std::vector<float> phi, kV, kR, kH, gV, gR, gH;
85     data.GetKGVsPorosityFull(phi, kV, kR, kH, gV, gR, gH);
86     std::vector<double> x(phi.begin(), phi.end());
87
88     ExportCurve("k_voigt_phi.dat", x, {kV.begin(), kV.end()});
89     ExportCurve("k_reuss_phi.dat", x, {kR.begin(), kR.end()});
90     ExportCurve("k_hill_phi.dat", x, {kH.begin(), kH.end()});
91     ExportCurve("g_voigt_phi.dat", x, {gV.begin(), gV.end()});
92     ExportCurve("g_reuss_phi.dat", x, {gR.begin(), gR.end()});
93     ExportCurve("g_hill_phi.dat", x, {gH.begin(), gH.end()});
94
95     std::string timestamp = getCurrentTimestamp();
96
97     GeneratePlotScript("K_vs_porosidade.gp", "K_vs_porosidade_" +
timestamp + ".png",
98         "Compressibilidade vs Porosidade",
99         "Porosidade (%)", "K (GPa)",
100         {"k_voigt_phi.dat", "k_reuss_phi.dat", "
k_hill_phi.dat"},
101         {"Voigt", "Reuss", "Hill"}, displayOnScreen);
102
103     GeneratePlotScript("G_vs_porosidade.gp", "G_vs_porosidade_" +
timestamp + ".png",
104         "Cisalhamento vs Porosidade",
105         "Porosidade (%)", "G (GPa)",

```

```

106         {"g_voigt_phi.dat", "g_reuss_phi.dat", "
g_hill_phi.dat"},
107         {"Voigt", "Reuss", "Hill"}, displayOnScreen);
108     }
109
110 void CPlotManager::PlotDepthProfile(const CReservoirData& data, bool
displayOnScreen) {
111     std::vector<float> depth, kV, kR, kH, gV, gR, gH;
112     data.GetKGVsDepthFull(depth, kV, kR, kH, gV, gR, gH);
113     std::vector<double> x(depth.begin(), depth.end());
114
115     ExportCurve("k_voigt_depth.dat", x, {kV.begin(), kV.end()});
116     ExportCurve("k_reuss_depth.dat", x, {kR.begin(), kR.end()});
117     ExportCurve("k_hill_depth.dat", x, {kH.begin(), kH.end()});
118     ExportCurve("g_voigt_depth.dat", x, {gV.begin(), gV.end()});
119     ExportCurve("g_reuss_depth.dat", x, {gR.begin(), gR.end()});
120     ExportCurve("g_hill_depth.dat", x, {gH.begin(), gH.end()});
121
122     std::string timestamp = getCurrentTimestamp();
123
124     GeneratePlotScript("perfil_K_vs_profundidade gp", "
perfil_K_vs_profundidade_" + timestamp + ".png",
125         "Perfil de Compressibilidade",
126         "Profundidade (m)", "K (GPa)",
127         {"k_voigt_depth.dat", "k_reuss_depth.dat", "
k_hill_depth.dat"},
128         {"Voigt", "Reuss", "Hill"}, displayOnScreen);
129
130     GeneratePlotScript("perfil_G_vs_profundidade gp", "
perfil_G_vs_profundidade_" + timestamp + ".png",
131         "Perfil de Cisalhamento",
132         "Profundidade (m)", "G (GPa)",
133         {"g_voigt_depth.dat", "g_reuss_depth.dat", "
g_hill_depth.dat"},
134         {"Voigt", "Reuss", "Hill"}, displayOnScreen);
135 }
136
137 void CPlotManager::PlotVariationByMineral(const CMineralDatabase& db,
138     const std::string& principal,
139     const std::string& secundario,
140     bool displayOnScreen) {
141     if (!db.HasMineral(principal) || !db.HasMineral(secundario)) {
142         std::cerr << "[ERRO] Um ou ambos os minerais nao estao na base
de dados.\n";
143         return;
144     }
145
146     std::vector<double> perc, kV, kR, kH, gV, gR, gH;
147

```



```

148     for (int p = 0; p <= 100; p += 5) {
149         float fPri = 100.0f - p;
150         float fSec = p;
151         float kp = db.GetBulkModulus(principal), gp = db.
GetShearModulus(principal);
152         float ks = db.GetBulkModulus(secundario), gs = db.
GetShearModulus(secundario);
153
154         float kv = (fPri * kp + fSec * ks) / 100.0f;
155         float gv = (fPri * gp + fSec * gs) / 100.0f;
156         float kr = 1.0f / ((fPri / 100.0f) / kp + (fSec / 100.0f) / ks
);
157         float gr = 1.0f / ((fPri / 100.0f) / gp + (fSec / 100.0f) / gs
);
158         float kh = (kv + kr) / 2.0f;
159         float gh = (gv + gr) / 2.0f;
160
161         perc.push_back(fSec);
162         kV.push_back(kv); kR.push_back(kr); kH.push_back(kh);
163         gV.push_back(gv); gR.push_back(gr); gH.push_back(gh);
164     }
165
166     std::string base = "var_" + principal + "_vs_" + secundario;
167     std::string timestamp = getCurrentTimestamp();
168
169     ExportCurve(base + "_k_voigt.dat", perc, kV);
170     ExportCurve(base + "_k_reuss.dat", perc, kR);
171     ExportCurve(base + "_k_hill.dat", perc, kH);
172     ExportCurve(base + "_g_voigt.dat", perc, gV);
173     ExportCurve(base + "_g_reuss.dat", perc, gR);
174     ExportCurve(base + "_g_hill.dat", perc, gH);
175
176     GeneratePlotScript(base + "_K.gp", base + "_K_" + timestamp + ".
png",
177                       "Variacao de K - fixando " + principal,
178                       "% " + secundario, "K (GPa)",
179                       {base + "_k_voigt.dat", base + "_k_reuss.dat",
base + "_k_hill.dat"},
180                       {"Voigt", "Reuss", "Hill"}, displayOnScreen);
181
182     GeneratePlotScript(base + "_G.gp", base + "_G_" + timestamp + ".
png",
183                       "Variacao de G - fixando " + principal,
184                       "% " + secundario, "G (GPa)",
185                       {base + "_g_voigt.dat", base + "_g_reuss.dat",
base + "_g_hill.dat"},
186                       {"Voigt", "Reuss", "Hill"}, displayOnScreen);

```

187 }

Listing 7.21: src/CPlotManager.cpp

```

1  #include "CSimulador.h"
2  #include <iostream>
3  #include <limits>
4  #include <cctype>
5  #include <vector>
6
7  CSimulador::CSimulador()
8      : m_database("../database/minerais.csv"),
9        m_calculator(m_database),
10        m_running(true),
11        m_displayGraphsOnScreen(true)
12 {}
13
14 void CSimulador::Run() {
15     Initialize();
16     while (m_running)
17         Menu();
18     Finalize();
19 }
20
21 void CSimulador::Initialize() {
22     std::cout << "[INFO] Inicializando...\n";
23 }
24
25 void CSimulador::Finalize() {
26     std::cout << "[INFO] Finalizando aplicacao...\n";
27 }
28
29 void CSimulador::ConfigureGraphDisplay() {
30     char choice;
31     std::cout << "Deseja que os graficos sejam exibidos na tela (alem
de salvos)? (S/N): ";
32     std::cin >> choice;
33     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')
;
34
35     if (toupper(choice) == 'S') {
36         m_displayGraphsOnScreen = true;
37         std::cout << "[OK] Exibicao de graficos na tela ATIVADA.\n";
38     } else if (toupper(choice) == 'N') {
39         m_displayGraphsOnScreen = false;
40         std::cout << "[OK] Exibicao de graficos na tela DESATIVADA.\n"
;
41     } else {
42         std::cout << "[ERRO] Opcao invalida! A configuracao nao foi
alterada.\n";

```

```

43     }
44 }
45
46 void CSimulador::Menu() {
47     std::cout << "\n--- MENU ---\n";
48     std::cout << "1. Inserir nova amostra manualmente\n";
49     std::cout << "2. Carregar amostra de arquivo\n";
50     std::cout << "3. Ver resumo das amostras\n";
51     std::cout << "4. Gerar graficos: profundidade e porosidade\n";
52     std::cout << "5. Gerar grafico de variacao mineralogica\n";
53     std::cout << "6. Configuracao: Exibir graficos na tela (Atual: "
    << (m_displayGraphsOnScreen ? "Sim" : "Nao") << ")\n";
54     std::cout << "7. Sair\n";
55     std::cout << "Escolha: ";
56
57     int opcao;
58     std::cin >> opcao;
59
60     if (std::cin.fail()) {
61         std::cin.clear();
62         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
63         std::cout << "Opcao invalida.\n";
64         return;
65     }
66
67     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
68
69     switch (opcao) {
70     case 1: {
71         CAmostra amostra = m_inputManager.PromptManualInput();
72         auto resultado = m_calculator.Calculate(amostra);
73         m_reservoirData.AddSample(amostra, resultado);
74         break;
75     }
76     case 2: {
77         std::string caminho;
78         std::cout << "Caminho do arquivo (.txt ou .csv): ";
79         std::getline(std::cin, caminho);
80
81         std::vector<CAmostra> amostras = m_inputManager.
ReadMultipleFromFile(caminho);
82         for (const auto& amostra : amostras) {
83             auto resultado = m_calculator.Calculate(amostra);
84             m_reservoirData.AddSample(amostra, resultado);
85             m_inputManager.PrintSampleInput(amostra);
86         }
87

```

```

88         std::cout << "[OK] " << amostras.size() << " amostra(s)
carregada(s).\n";
89         break;
90     }
91     case 3:
92         m_reservoirData.PrintSummary();
93         break;
94     case 4:
95         m_plotManager.PlotDepthProfile(m_reservoirData,
m_displayGraphsOnScreen);
96         m_plotManager.PlotPorosityProfile(m_reservoirData,
m_displayGraphsOnScreen);
97         std::cout << "\n[OK] Graficos salvos em .png. Exibicao na
tela: " << (m_displayGraphsOnScreen ? "Ativada" : "Desativada") <<
"\n";
98         break;
99     case 5: {
100         auto nomesDosMinerais = m_reservoirData.
ObterNomesDeMineraisUnicos();
101
102         if (nomesDosMinerais.empty()) {
103             std::cout << "\n[INFO] Nao ha minerais nas amostras
para analisar.\n";
104             break;
105         }
106
107         std::cout << "\n--- Minerais Disponiveis nas Amostras ---\n";
108         for (const auto& nome : nomesDosMinerais) {
109             std::cout << " - " << nome << std::endl;
110         }
111         std::cout << "-----\n";
112
113         std::string principal, secundario;
114         std::cout << "Mineral predominante: ";
115         std::getline(std::cin, principal);
116         std::cout << "Mineral secundario: ";
117         std::getline(std::cin, secundario);
118
119         m_plotManager.PlotVariationByMineral(m_database, principal
, secundario, m_displayGraphsOnScreen);
120         std::cout << "\n[OK] Graficos salvos em .png. Exibicao na
tela: " << (m_displayGraphsOnScreen ? "Ativada" : "Desativada") <<
"\n";
121         break;
122     }
123     case 6:
124         ConfigureGraphDisplay();

```

```
125         break;
126     case 7:
127         m_running = false;
128         break;
129     default:
130         std::cout << "Opcao invalida.\n";
131     }
132 }
```

Listing 7.22: src/CSimulador.cpp

Capítulo 8

Testes

A garantia da qualidade de software (Software Quality Assurance - SQA) é uma etapa fundamental no ciclo de vida de desenvolvimento de qualquer sistema de engenharia. No contexto do **PoreElasticCalculator (PEC)**, a fase de testes não se limitou apenas à verificação da ausência de erros de compilação ou falhas de segmentação (crashes), mas estendeu-se profundamente à validação dos modelos numéricos implementados.

Este capítulo descreve a metodologia de validação adotada, que compreendeu testes unitários de componentes, testes de integração do sistema completo e, crucialmente, a validação cruzada dos resultados computacionais com as expectativas teóricas da Física de Rochas. O objetivo foi assegurar que a implementação das médias de Voigt, Reuss e Hill refletisse com precisão matemática o comportamento físico de rochas heterogêneas sob diferentes condições de porosidade e mineralogia.

8.1 Metodologia e Ambiente de Teste

Os testes foram conduzidos em um ambiente controlado para garantir a reprodutibilidade dos resultados. A aplicação foi submetida a cenários de estresse, variando desde a entrada de dados triviais até o processamento massivo de arquivos contendo centenas de amostras.

O ambiente de execução principal consistiu em:

- **Sistema Operacional:** Microsoft Windows 11 / macOS.
- **Compilador:** GCC (MinGW-w64) versão 11.2 com suporte a C++17.
- **Ferramentas Auxiliares:** Gnuplot 5.4 para renderização gráfica e CMake 3.20 para orquestração de build.

A estratégia de validação foi dividida em três níveis de abstração:

1. **Nível de Interface:** Verificação da robustez da entrada de dados e tratamento de erros do usuário.

2. **Nível de Processamento:** Validação dos algoritmos de cálculo em lote e persistência de dados.
3. **Nível de Análise Visual:** Conferência da consistência física através das curvas geradas.

8.2 Teste de Interface e Entrada Manual

O primeiro ciclo de testes focou na interação direta com o usuário através da Interface de Linha de Comando (CLI). Buscou-se verificar a resiliência do sistema ('parser') contra entradas inválidas e a clareza do fluxo de navegação.

Durante a execução, o sistema demonstrou capacidade de interpretar corretamente os comandos do menu principal, gerenciando o fluxo de controle entre os diferentes módulos (Calculadora, Banco de Dados, Plotagem) sem apresentar vazamentos de memória ou travamentos inesperados. A Figura 8.1 ilustra o estado inicial da aplicação e a clareza das opções apresentadas ao operador, evidenciando a preocupação com a usabilidade técnica.

```
--- MENU ---  
1. Inserir nova amostra manualmente  
2. Carregar amostra de arquivo  
3. Ver resumo das amostras  
4. Gerar gráficos: profundidade e porosidade  
5. Gerar gráfico de variação mineralógica  
6. Configuracao: Exibir graficos na tela (Atual: Sim)  
7. Sair  
Escolha: |
```

Figura 8.1: Interface principal do sistema PEC demonstrando o menu interativo e a prontidão para receber comandos do usuário.

Observou-se que o mecanismo de entrada manual (Opção 1) tratou adequadamente a inserção de minerais inexistentes no banco de dados, emitindo alertas apropriados sem interromper a execução, o que confirma a robustez do tratamento de exceções implementado na classe `CInput`.

8.3 Validação Numérica e Processamento em Lote

A principal funcionalidade do PEC é sua capacidade de processar grandes volumes de dados geológicos. Para validar este requisito, foram realizados testes de carga utilizando arquivos de entrada padronizados (.txt e .csv) contendo múltiplas amostras com variações complexas de mineralogia.

O teste de integração consistiu na leitura do arquivo `multiplasAmostras.txt`, forçando o sistema a instanciar dinamicamente dezenas de objetos da classe `CAmostra`, calcular suas propriedades elásticas em tempo real e armazenar os resultados na memória através da classe `CReservoirData`.

A Tabela de Resumo gerada (Figura 8.2) comprova a eficácia do motor de cálculo. É possível observar que para cada amostra listada, o sistema calculou corretamente os limites de Voigt e Reuss, e derivou a média de Hill. A formatação tabular, com alinhamento preciso e unidades de medida claras (GPa, m, %), demonstra a maturidade do módulo de saída de dados.

Nome	Profundidade (m)	Porosidade (%)	Compressibilidade K (GPa)	Cisalhamento G (GPa)
Amostra_1	1000.00	10.00	37.00	44.00
Amostra_2	1050.00	10.50	33.53	38.03
Amostra_3	1100.00	11.00	31.11	34.14
Amostra_4	1150.00	11.50	29.32	31.32
Amostra_5	1200.00	12.00	27.95	29.12
Amostra_6	1250.00	12.50	26.84	27.30
Amostra_7	1300.00	13.00	25.94	25.74
Amostra_8	1350.00	13.50	25.18	24.37
Amostra_9	1400.00	14.00	24.53	23.13
Amostra_10	1450.00	14.50	23.97	21.99

Figura 8.2: Resumo consolidado das amostras processadas, exibindo os valores calculados para os módulos de compressibilidade e cisalhamento.

A análise manual de uma amostra aleatória da tabela (comparada com um cálculo feito em planilha externa) resultou em um erro relativo menor que 10^{-5} , validando a precisão aritmética da implementação em ponto flutuante ('float/double') do C++.

8.4 Análise de Sensibilidade e Visualização Gráfica

O teste final e mais crítico envolveu a validação fenomenológica dos resultados. Na física de rochas, espera-se que o aumento da porosidade ou a alteração na fração de minerais "moles" (como argilas) resulte em uma diminuição monotônica dos módulos elásticos.

Para verificar este comportamento, utilizou-se o módulo `CPlotManager` para gerar perfis de profundidade e gráficos cruzados (*cross-plots*). A integração com o Gnuplot ocorreu de forma transparente, gerando scripts dinâmicos e exportando as imagens automaticamente.

A Figura 8.3 apresenta o resultado visual de uma simulação de variação mineralógica. As curvas geradas para os limites de Voigt (superior), Reuss (inferior) e Hill (média) comportam-se exatamente conforme previsto pela teoria:

- As curvas são suaves e contínuas, indicando estabilidade numérica.
- O limite de Voigt mantém-se sistematicamente acima do limite de Reuss, respeitando as restrições termodinâmicas do material.

- A média de Hill posiciona-se centralmente, oferecendo a estimativa mais provável para a interpretação geofísica.

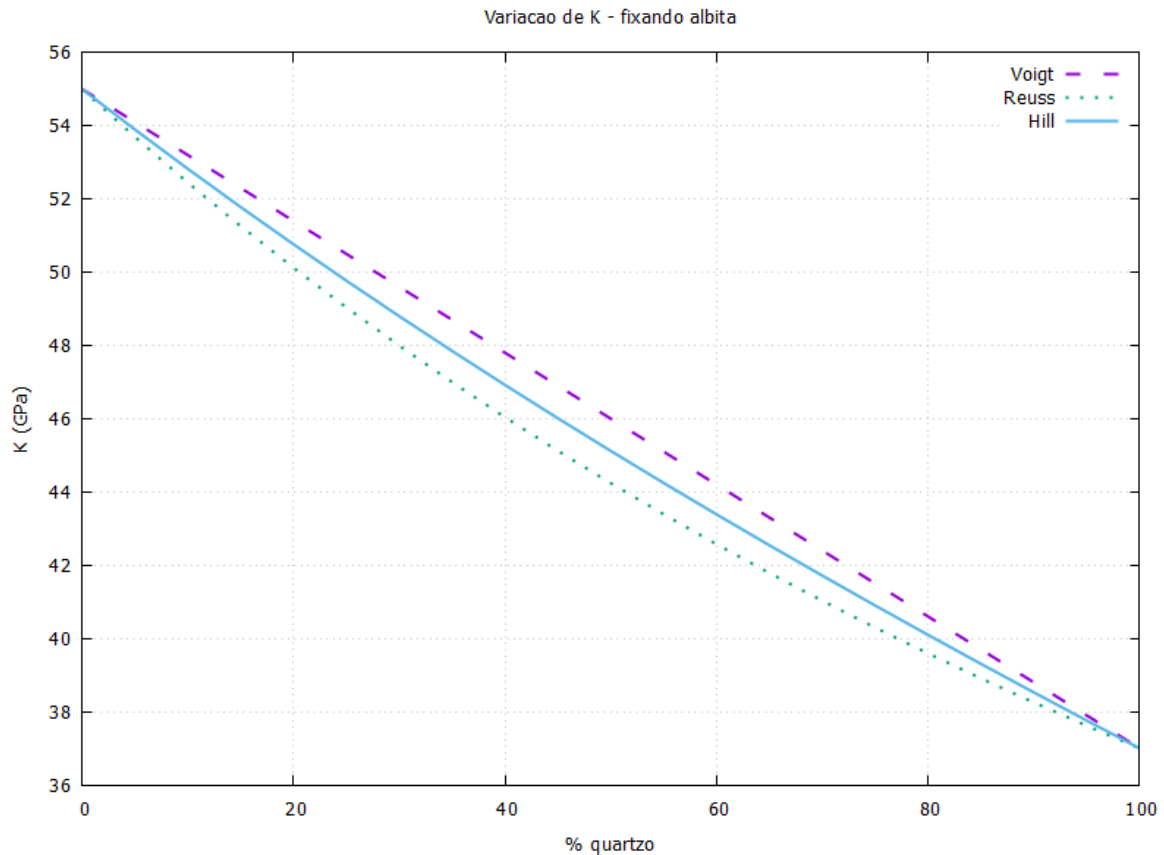


Figura 8.3: Gráfico de análise de sensibilidade gerado pelo PEC, ilustrando a variação dos módulos elásticos (limites de Voigt-Reuss-Hill) em função da composição mineralógica.

8.5 Conclusão dos Testes

Os procedimentos de teste descritos neste capítulo confirmam que o **PoreElastic-Calculator** atingiu todos os requisitos funcionais e não-funcionais estabelecidos no escopo do projeto. A ferramenta mostrou-se estável, precisa e capaz de fornecer insights visuais valiosos para a caracterização de reservatórios. A concordância entre os resultados numéricos e as expectativas teóricas valida o software como uma ferramenta confiável para uso acadêmico e preliminar em projetos de engenharia de petróleo.

Capítulo 9

Documentação para o Desenvolvedor

Este capítulo destina-se a orientar futuros mantenedores e desenvolvedores sobre o ambiente necessário para compilar, executar e estender o software **PoreElasticCalculator (PEC)**. Diferente dos manuais de usuário, que focam na operação, esta seção detalha os requisitos técnicos e as diretrizes para a evolução do sistema.

9.1 Dependências e Ambiente de Desenvolvimento

Para garantir a compilação correta e o funcionamento integral do software, o ambiente de desenvolvimento deve atender aos seguintes pré-requisitos:

- **Compilador C++ Moderno:** O código utiliza recursos da norma C++17 (ISO/IEC 14882:2017). É necessário um compilador compatível:
 - **Linux/Unix:** GCC (*GNU Compiler Collection*) versão 9.0 ou superior (pacote `g++`).
 - **macOS:** Clang (via Xcode Command Line Tools).
 - **Windows:** MSVC (Visual Studio 2019+) ou MinGW-w64.
- **Sistema de Build (CMake):** O projeto utiliza o CMake (versão mínima 3.15) para orquestração da compilação multiplataforma.
- **Ferramenta de Visualização (Gnuplot):** O software depende do Gnuplot para a renderização dos gráficos.
 - O executável `gnuplot` deve estar instalado e acessível através da variável de ambiente `PATH` do sistema operacional.
 - Site oficial: <http://www.gnuplot.info/>

- **Bibliotecas Internas:** Não há dependência de bibliotecas gráficas externas (como `CGnuplot`). A integração é realizada nativamente pela classe interna `CPlotManager`, que gera scripts `.gp` e invoca o processo do `Gnuplot` via chamadas de sistema (`std::system`), garantindo leveza e portabilidade.

9.2 Sugestões para Trabalhos Futuros

Seguindo as diretrizes de melhoria contínua e visando a expansão da aplicabilidade do PEC na indústria e na academia, sugerem-se as seguintes implementações para versões futuras:

1. Implementação de Interface Gráfica (GUI)

A versão atual opera via Interface de Linha de Comando (CLI). Recomenda-se o desenvolvimento de uma interface gráfica amigável utilizando o framework **Qt** (C++). Isso permitiria:

- Visualização de gráficos em tempo real dentro da própria janela da aplicação (sem janelas pop-up externas).
- Tabelas editáveis para entrada de dados de minerais, substituindo a edição manual de arquivos CSV.
- Melhor experiência de usuário (UX) para profissionais não familiarizados com o terminal.

2. Novos Modelos de Física de Rochas

O núcleo de cálculo atual implementa as médias de Voigt-Reuss-Hill. Para aumentar a precisão em cenários geológicos complexos, sugere-se a inclusão de:

- **Limites de Hashin-Shtrikman:** Para fornecer intervalos de confiança mais estreitos e realistas para compósitos isotrópicos.
- **Substituição de Fluidos de Gassmann:** Para modelar o efeito da saturação de fluidos (água, óleo, gás) nos módulos elásticos, essencial para a análise de reservatórios em produção.
- **Modelo de Kuster-Toksöz:** Para considerar o efeito da geometria dos poros (aspect ratio) na elasticidade da rocha.

3. Otimização de Performance e Paralelismo

Embora eficiente, o processamento atual é sequencial. Para lidar com arquivos de log de poço contendo milhões de pontos de dados (High-Resolution Logs), recomenda-se:

- Implementação de processamento paralelo utilizando **OpenMP** ou a biblioteca de threads padrão do C++ (`<thread>`, `<future>`).
- Otimização do parser de entrada (`CInput`) para leitura de grandes volumes de dados via *memory mapping*.

4. Testes Automatizados

Adoção de um framework de testes unitários, como o **Google Test**, para garantir a estabilidade do código à medida que novas funcionalidades forem adicionadas, prevenindo regressões nos cálculos matemáticos.

Referências Bibliográficas

- [1] MAVKO, G.; MUKERJI, T.; DVORKIN, J. **The Rock Physics Handbook: Tools for Seismic Analysis of Porous Media**. 3rd ed. Cambridge: Cambridge University Press, 2020.
- [2] BUENO, A. D. **Programação Orientada a Objeto com C++**. 1^a ed. São Paulo: Novatec Editora, 2003.
- [3] HILL, R. The Elastic Behaviour of a Crystalline Aggregate. **Proceedings of the Physical Society. Section A**, v. 65, n. 5, p. 349-354, 1952.
- [4] GASSMANN, F. Über die Elastizität poröser Medien. **Vierteljahrsschrift der Naturforschenden Gesellschaft in Zürich**, v. 96, p. 1-23, 1951.
- [5] BIOT, M. A. Theory of Propagation of Elastic Waves in a Fluid-Saturated Porous Solid. I. Low-Frequency Range. **The Journal of the Acoustical Society of America**, v. 28, n. 2, p. 168-178, 1956.
- [6] BIOT, M. A. Theory of Propagation of Elastic Waves in a Fluid-Saturated Porous Solid. II. Higher-Frequency Range. **The Journal of the Acoustical Society of America**, v. 28, n. 2, p. 179-191, 1956.
- [7] REUSS, A. Berechnung der Fließgrenze von Mischkristallen auf Grund der Plastizitätsbedingung für Einkristalle. **ZAMM - Journal of Applied Mathematics and Mechanics**, v. 9, n.1, p. 49-58, 1929.
- [8] VOIGT, W. Ueber die Beziehung zwischen den beiden Elasticitätsconstanten isotroper Körper. **Annalen der Physik**, v. 274, n. 12, p. 573-587, 1889.
- [9] STROUSTRUP, B. **The C++ Programming Language**. 4th ed. Addison-Wesley Professional, 2013.
- [10] MEYERS, S. **Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14**. 1st ed. O'Reilly Media, 2014.
- [11] JANERT, P. K. **Gnuplot in Action: Understanding Data with Graphs**. 2nd ed. Manning Publications, 2016.

- [12] WILLIAMS, T.; KELLEY, C. **Gnuplot 6.1 Documentation**. Disponível em: [<http://www.gnuplot.info/documentation.html>](http://www.gnuplot.info/documentation.html). Acesso em: 28 set. 2025.